CS336 作业 1：基础——语言建模（Basics: Language Modeling）

原文：Stanford CS336 作业说明（英文原版）

本文档为英文原版作业说明的非官方中文翻译，仅供学习参考；如与原文有歧义，请以英文原文为准。

目录

1 作业总览 **3**

2 字节对编码（BPE）分词器 **5**

2.1 Unicode 标准 5

2.2 Unicode 编码 6

2.3 子词分词 7

2.4 BPE 分词器的训练 8

2.5 BPE 分词器训练实验 10

2.6 BPE 分词器：编码与解码 12

2.6.1 编码文本 13

2.6.2 解码文本 13

2.7 实验 14

3 Transformer 语言模型架构 **15**

3.1 Transformer 语言模型 15

3.2 附注：批处理、Einsum 与高效计算 16

3.2.1 数学记法与内存顺序（Mathematical Notation and Memory Ordering） . . 19

3.3 基础构建模块：Linear 与 Embedding 模块 20

3.3.1 参数初始化（Parameter Initialization） 20

3.3.2 Linear 模块（Linear Module） 20

3.3.3 Embedding 模块（Embedding Module） 21

3.4 Pre-Norm Transformer 块 22

3.4.1 均方根层归一化（Root Mean Square Layer Normalization） 22
3.4.2 逐位置前馈网络（Position-Wise Feed-Forward Network） 23
3.4.3 相对位置嵌入（Relative Positional Embeddings） 25
3.4.4 缩放点积注意力（Scaled Dot-Product Attention） 26
3.4.5 因果多头自注意力（Causal Multi-Head Self-Attention） 27
3.5 完整的 Transformer 语言模型 28

4 训练 Transformer 语言模型 30
4.1 交叉熵损失（Cross-entropy loss） 31
4.2 SGD 优化器 31
4.2.1 在 PyTorch 中实现 SGD（Implementing SGD in PyTorch） 32
4.3 AdamW 33
4.4 学习率调度（Learning rate scheduling） 35
4.5 梯度裁剪（Gradient clipping） 36

5 训练循环（Training loop） 36
5.1 数据加载器（Data Loader） 37
5.2 检查点（Checkpointing） 38
5.3 训练循环（Training loop） 39

6 文本生成（Generating text） 39

7 实验（Experiments） 41
7.1 如何运行实验与交付要求（How to Run Experiments and Deliverables） 41
7.2 TinyStories 41
7.2.1 超参数调优（Hyperparameter tuning） 42
7.2.2 整合到一起（Putting it together） 42
7.2.3 调试模型架构的技巧与窍门（Tips and tricks for debugging model architectures） 42
7.3 消融实验与架构修改（Ablations and architecture modification） 45
7.4 在 OpenWebText 上运行（Running on OpenWebText） 46
7.5 你自己的改进 + 排行榜（Your own modification + leaderboard） 47

1 作业总览

在本作业中，你将构建训练一个标准 Transformer 语言模型（language model，LM）所需的全部组件，并训练若干模型。

你将实现的内容

1. 字节对编码（byte-pair encoding，BPE）分词器（见第 2 节）
2. Transformer 语言模型（LM）（见第 3 节）
3. 交叉熵损失函数与 AdamW 优化器（见第 4 节）
4. 训练循环，支持模型与优化器状态的序列化保存和加载（见第 5 节）

你将运行的内容

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 用训练好的分词器对数据集进行编码，把它转换为整数 ID 序列。
3. 在 TinyStories 数据集上训练 Transformer 语言模型。
4. 使用训练好的 Transformer 语言模型进行样本生成与困惑度（perplexity）评估。
5. 在 OpenWebText 上训练模型，并把你取得的困惑度提交到排行榜（leaderboard）。

你可以使用的内容

我们期望你从零开始构建每个组件。特别地，除非属于下面列出的内容，你不得使用 `torch.nn` 、`torch.nn.functional` 或 `torch.optim` 中的任何定义：

- `torch.nn.Parameter`
- `torch.nn` 中的容器类（例如 `Module` 、`ModuleList` 、`Sequential` 等）。[1]
- `torch.optim.Optimizer` 基类

除此之外的任何 PyTorch 定义你都可以使用。如果你想使用某个函数或类，但不确定它是否被允许，欢迎随时在 Slack 上提问。拿不准的时候，请思考一下：使用它是否会损害本作业” 从零实现” 的初衷。

关于 AI 工具的声明

AI 可以完全自主地解决作业中的许多部分。这使得深入投入课程材料并从中学习变得更加困难。

允许将 AI 工具用于回答高层次的概念性问题，或提供低层次的编程文档（例如函数签名、库 API）。但是，不允许用 AI 工具实现作业的任何部分。这既包括编码智能体（例如 Cursor Agents、

[1] 完整列表见 pytorch.org/docs/stable/nn.html#containers。

Codex、Claude Code），也包括 AI 自动补全（例如 Cursor Tab、GitHub Copilot）。使用 AI 智能体时，请确保它使用随作业提供的 `AGENTS.md` 文件；使用聊天机器人时，也请附上你使用的提示词（prompt）。

我们强烈建议你在完成作业时在 IDE 中禁用 AI 自动补全（例如 Cursor Tab、GitHub Copilot）（不过非 AI 的自动补全，例如函数名自动补全，完全没有问题）。往届同学特别提到，禁用 AI 自动补全让他们更容易深入理解课程内容。

完整的 AI 政策请参见这份文档（链接见原文）。

代码是什么样的

作业代码以及本说明文档都发布在 GitHub 上：

github.com/stanford-cs336/assignment1-basics

请 `git clone` 该仓库。如果仓库有更新，我们会通知你，届时你 `git pull` 即可获得最新内容。

1. `cs336_basics/*`：这是你写代码的地方。注意这里面没有任何代码——你完全可以从零开始自由发挥！
2. `adapters.py`：这里定义了你的代码必须提供的一组功能。对于每一项功能（例如缩放点积注意力），请在对应的实现函数（例如 `run_scaled_dot_product_attention`）里直接调用你自己的代码来完成填充。注意：你对 `adapters.py` 的修改不应包含任何实质性逻辑；它只是胶水代码。
3. `test_*.py`：这里包含你必须通过的全部测试（例如 `test_scaled_dot_product_attention`），这些测试会调用 `adapters.py` 中定义的钩子（hook）。请不要编辑测试文件。

如何提交

要提交，请运行 `make_submission.sh` 生成提交用的 zip 文件。如果你有不希望包含在提交 zip 里的较大数据文件或 checkpoint，请务必把它们添加到脚本中的排除列表。

你需要在 Gradescope 上提交以下文件：

- `writeup.pdf`：回答所有书面问题。请对你的回答进行排版。
- `code.zip`：包含你编写的全部代码。

要提交到排行榜，请向以下仓库提交 PR：

github.com/stanford-cs336/assignment1-basics-leaderboard

详细的提交说明见排行榜仓库中的 `README.md`。

从哪里获取数据集

本作业将使用两个预处理数据集：TinyStories [R. Eldan et al., 2023] 与 OpenWebText [A. Gokaslan et al., 2019]。这两个数据集都是单个的大型纯文本文件。

如果你是随课程一起完成本作业，可以在 compute guide 中找到数据集的下载说明。

如果你是在家自学，可以使用 `README.md` 中的命令下载这些文件。

低资源提示：初始化（Init）

在本课程的各次作业说明中，我们会给出在较少乃至没有 GPU 资源的情况下完成作业的建议。例如，我们有时会建议**缩减规模**（downscaling）你的数据集或模型大小，或者讲解如何在 Mac 的集成 GPU 或 CPU 上运行训练代码。你会在这类蓝色方框中看到这些" 低资源提示"。即便是可以使用课程机器的斯坦福在校生，这些提示也可能帮助你更快地迭代并节省时间，因此我们建议阅读它们！

低资源提示：在 Apple Silicon 或 CPU 上完成作业 1

使用助教（staff）的参考解答代码，我们可以在配备 36 GB 内存的 Apple M4 Max 芯片上训练出一个能生成相当流畅文本的 LM：在 Metal GPU（MPS）上用时不到 5 分钟，用 CPU 大约 30 分钟。如果这些说法对你来说没什么概念，别担心！你只需要知道：只要你的笔记本电脑还算新，且你的实现正确而高效，你就能训练出一个能以不错的流畅度生成简单儿童故事的小型 LM。

在作业后面的部分，我们会说明如果你使用的是 CPU 或 MPS，需要做哪些改动。

2 字节对编码（BPE）分词器

在作业的第一部分，我们将训练并实现一个字节级的字节对编码（BPE）分词器 [R. Sennrich et al., 2016; C. Wang et al., 2019]。具体来说，我们会把任意（Unicode）字符串表示为字节序列，并在该字节序列上训练 BPE 分词器。之后，我们会用这个分词器把文本（字符串）编码为 token（整数序列），用于语言建模。

2.1 Unicode 标准

Unicode 是一种文本编码标准，它把字符映射为整数码点（code point）。截至 Unicode 17.0（2025 年 9 月发布），该标准在 172 个文字系统（script）中定义了 159,801 个字符。例如，字符"s" 的码点是 115（通常记作 U+0073，其中 U+ 是约定前缀，0073 是 115 的十六进制表示），而字符" 牛" 的码点是 29275。在 Python 中，你可以用 `ord()` 函数把单个 Unicode 字符转换为它的整数表示；`chr()` 函数则把一个整数 Unicode 码点转换为含有对应字符的字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

问题（unicode1 ）：理解 Unicode（1 分）

(a) chr(0) 返回的是什么 Unicode 字符？

交付要求：一句话回答。

(b) 这个字符的字符串表示（__repr__() ）与它的打印（print）表示有什么不同？

交付要求：一句话回答。

(c) 当这个字符出现在文本中时会发生什么？你可以在 Python 解释器里试着运行下面的代码，看看结果是否符合你的预期：

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

交付要求：一句话回答。

2.2 Unicode 编码

虽然 Unicode 标准定义了从字符到码点（整数）的映射，但直接在 Unicode 码点上训练分词器并不现实：这样的词表会过于庞大（约 15 万项）且稀疏（许多字符都非常罕见）。因此我们改用 Unicode 编码（Unicode encoding），它把一个 Unicode 字符转换为字节序列。Unicode 标准本身定义了三种编码：UTF-8、UTF-16 和 UTF-32，其中 UTF-8 是互联网上的主导编码（超过 98% 的网页都在使用）。

要把 Unicode 字符串编码为 UTF-8，可以使用 Python 的 encode() 函数。要访问 Python bytes 对象的底层字节值，可以对其进行迭代（例如调用 list() ）。最后，可以用 decode() 函数把 UTF-8 字节串解码为 Unicode 字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # 取得编码后字符串的字节值（0 到 255 的整数）。
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # 一个字节并不一定对应一个 Unicode 字符!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

把 Unicode 码点转换为字节序列（例如通过 UTF-8 编码）后，我们实际上是把一个码点序列（21 位整数，其中有 159,801 个有效值）转换成了一个字节值序列（0 到 255 范围内的整数）。长度为 256 的字节词表处理起来要容易得多。使用字节级分词（byte-level tokenization）时，我们不需要担心词表外（out-of-vocabulary）token，因为我们知道任何输入文本都可以表示为 0 到 255 的整数序列。

问题（unicode2 ）：Unicode 编码（3 分）

(a) 相比 UTF-16 或 UTF-32，为什么更倾向于在 UTF-8 编码的字节上训练我们的分词器？比较这些编码对各种输入字符串的输出可能会对你有所帮助。

交付要求：一到两句话回答。

(b) 考虑下面这个（不正确的）函数，它的本意是把 UTF-8 字节串解码为 Unicode 字符串。这个函数为什么是不正确的？请给出一个会产生错误结果的输入字节串示例。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

交付要求：给出一个使 `decode_utf8_bytes_to_str_wrong` 产生错误输出的输入字节串示例，并用一句话解释该函数为什么是不正确的。

(c) 给出一个无法解码为任何 Unicode 字符的两字节序列。

交付要求：给出一个示例，并用一句话解释。

2.3 子词分词

字节级分词虽然能缓解词级分词器面临的词表外问题，但把文本切分为字节会导致输入序列极长。这会拖慢模型训练：一个含 10 个单词的句子在词级语言模型中可能只有 10 个 token 长，而在字符级模型中可能有 50 个甚至更多 token（取决于单词的长度）。处理这些更长的序列需要模型在每一步进行更多计算。此外，在字节序列上做语言建模也很困难，因为更长的输入序列会在数据中引入长程依赖。

子词分词（subword tokenization）是介于词级分词器与字节级分词器之间的一种折中。注意，字节级分词器的词表有 256 个条目（字节取值为 0 到 255）。子词分词器以更大的词表规模换取对输入字节序列更好的压缩。例如，如果字节序列 `b'the'` 在我们的原始文本训练数据中频繁出现，就在词表中为它分配一个条目，这样这个 3-token 序列就可以缩减为单个 token。

我们如何挑选这些要加入词表的子词单元？R. Sennrich et al. [3] 提出使用字节对编码（BPE；P. Gage [5]），它是一种压缩算法，迭代地用一个新的、未被使用的索引来替换（”合并”）出现频率最高的字节对。注意，该算法通过向词表添加子词 token 来最大化输入序列的压缩——只要一个词在输入文本中出现得足够频繁，它最终就会被表示为单个子词单元。

词表通过 BPE 构建的子词分词器通常称为 BPE 分词器。在本作业中，我们将实现一个字节级 BPE 分词器，其词表条目是字节或合并后的字节序列，这在词表外处理与可控的输入序列长度两方面兼得。构建 BPE 分词器词表的过程称为” 训练”BPE 分词器。

2.4 BPE 分词器的训练

BPE 分词器的训练流程包括三个主要步骤。

词表初始化

分词器词表是从字节串（bytestring）token 到整数 ID 的一一映射。由于我们训练的是字节级 BPE 分词器，初始词表就是全部字节的集合。因为字节取值共有 256 种，所以初始词表的大小为 256。

预分词

有了词表之后，原则上你可以直接统计文本中各字节相邻出现的频率，并从最高频的字节对开始合并。然而这在计算上相当昂贵，因为每合并一次都要对语料库做一次完整扫描。此外，直接在整个语料库上跨边界合并字节，可能产生仅标点不同的 token（例如 `dog!` 与 `dog.` ）。这些 token 会被赋予完全不同的 token ID，尽管它们很可能具有很高的语义相似性（因为它们只差一个标点）。

为了避免这一点，我们对语料库进行预分词（pre-tokenize）。你可以把它理解为对语料库的一种粗粒度分词，它帮助我们统计字符对出现的频率。例如，单词’text’ 可能是出现了 10 次的一个预分词（pre-token）。这样，当我们统计字符’t’ 和’e’ 相邻出现的次数时，会发现单词’text’ 中’t’ 与’e’ 相邻，于是可以把它们的计数直接加 10，而不必翻查整个语料库。由于我们训练的是字节级 BPE 模型，每个预分词都被表示为一个 UTF-8 字节序列。

R. Sennrich et al. [3] 的原始 BPE 实现直接按空白切分来预分词（即 `s.split(" ")` ）。基于 SentencePiece 的分词器（例如 Llama 1 和 2 的分词器）至今仍在使用这种方法。

大多数现代分词器使用基于正则表达式的预分词器，这一做法源自 GPT-2；A. Radford et al. [6]。我们将使用原始正则的一个略加修饰的版本，取自 github.com/openai/tiktoken/pull/234/files：

```
>>> PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

用这个预分词器交互式地切分一些文本，或许有助于你更好地理解它的行为：

```
>>> # 需要 `regex` 包
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

不过在代码中使用它时，你应当使用 `re.finditer` ，以免在构建” 预分词 → 计数” 的映射时把这些预分词得到的词都存储下来。

计算 BPE 合并

现在我们已经把输入文本转换为预分词，并把每个预分词表示为 UTF-8 字节序列，接下来就可以计算 BPE 合并（即训练 BPE 分词器）了。概括地说，BPE 算法迭代地统计每一对字节，并

找出频率最高的对（”A”, ”B”）。随后，这个最高频对（”A”, ”B”）的每一处出现都会被合并，即替换为新 token ”AB”。这个新合并出的 token 会被加入词表；因此 BPE 训练结束后的最终词表大小等于初始词表大小（在我们的设定中为 256）加上训练期间执行的 BPE 合并次数。为了提高 BPE 训练效率，我们不考虑跨越预分词边界的字节对。[2]在计算合并时，如果字节对的频率出现并列，则确定性地偏向字典序（lexicographically）更大的对。例如，如果（”A”, ”B”）、（”A”, ”C”）、（”B”, ”ZZ”）和（”BA”, ”A”）都具有最高频率，我们会合并（”BA”, ”A”）：

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

特殊 token

有些字符串（例如 `<|endoftext|>` ）常被用来编码元数据（例如文档之间的边界）。在编码文本时，通常需要把某些字符串当作” 特殊 token”（special tokens）对待：它们永远不应被切分成多个 token（即始终保留为单个 token）。例如，序列结束字符串 `<|endoftext|>` 应当始终保留为单个 token（即单个整数 ID），这样我们才知道什么时候应停止语言模型的生成。这些特殊 token 必须加入词表，从而拥有对应的固定 token ID。

R. Sennrich et al. [3] 的算法 1 给出了一个低效的 BPE 分词器训练实现（基本遵循我们上面概述的步骤）。作为第一个练习，实现并测试这个函数或许有助于检验你的理解。

示例（`bpe_example` ）：BPE 训练示例

这是取自 R. Sennrich et al. [3] 的一个风格化示例。考虑一个由以下文本构成的语料库

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

并且词表中有一个特殊 token `<|endoftext|>` 。

词表

我们用特殊 token `<|endoftext|>` 和 256 个字节值来初始化词表。

预分词

为了简单起见，也为了聚焦合并过程，本示例假设预分词就是按空白切分。预分词并计数后，我们得到如下频率表：

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

把它表示为 `dict[tuple[bytes, ...], int]` 会很方便，例如 `{(l,o,w): 5, ...}` 。注意，即便单个字节在 Python 中也是一个 `bytes` 对象。Python 中没有用来表示单个字节的 `byte` 类型，正如没有用来表示单个字符的 `char` 类型一样。

合并

[2]注意，R. Sennrich et al. [3] 的原始 BPE 公式规定要引入词尾（end-of-word）token。我们训练字节级 BPE 模型时不添加词尾 token，因为所有字节（包括空白和标点）都已包含在模型词表中。既然我们已经显式地表示了空格和标点，学到的 BPE 合并自然会反映出这些词边界。

我们先考察每一对相邻字节，并对它们出现的词的频率求和：{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6} 。字节对 ('e', 's') 与 ('s', 't') 频率并列，因此我们取字典序更大的 ('s', 't')。接着我们合并预分词，得到 {(l,o,w): 5, (l,o,w,e,r): 2, (w, i,d,e,st): 3, (n,e,w,e,st): 6} 。

第二轮中，我们发现 (e, st) 是最常见字节对（计数为 9），将其合并后得到 {(l,o,w): 5, (l,o,w, e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6} 。依此继续，最终得到的合并序列为 ['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r'] 。

如果我们只做 6 次合并，得到 ['s t', 'e st', 'o w', 'l ow', 'w est', 'n e'] ，此时词表条目为 [<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne] 。

使用这个词表和这组合并，单词 newest 会被分词为 [ne, west] 。

2.5 BPE 分词器训练实验

让我们在 TinyStories 数据集上训练一个字节级 BPE 分词器。数据集的查找/下载说明见第 1 节。在开始之前，我们建议先浏览一下 TinyStories 数据集，了解其中的数据大概是什么样子。

并行化预分词

你会发现预分词步骤是一个主要瓶颈。可以使用内置库 multiprocessing 对代码进行并行化来加速预分词。具体来说，我们建议在并行实现预分词时对语料库分块，并确保块边界落在某个特殊 token 的开头。你可以原封不动地使用下面链接中的起始代码来获得块边界，然后用它把工作分配到你的各个进程：

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

这种分块总是有效的，因为我们从不希望跨越文档边界进行合并。就本作业而言，你始终可以这样切分。不必担心收到一个不含 <|endoftext|> 的超大语料库这种边缘情况。

预分词前移除特殊 token

在用正则模式运行预分词（使用 re.finditer ）之前，你应当先把语料库（如果采用并行实现，则是你的分块）中的所有特殊 token 剔除。请确保按特殊 token 进行切分，使任何合并都不会跨越它们所界定的文本。例如，如果你的语料库（或分块）形如 [Doc 1]<|endoftext|>[Doc 2] ，你应当按特殊 token <|endoftext|> 切分，再分别对 [Doc 1] 和 [Doc 2] 预分词，这样合并就不会跨越文档边界。换句话说，特殊 token 在训练期间定义了硬性的分段边界，但它们本身不应计入合并统计。这可以通过 re.split 并以 "|".join(special_tokens) 作为分隔符来实现（注意小心使用 re.escape ，因为特殊 token 中可能出现 | 字符）。测试 test_train_bpe_special_tokens 会检验这一点。

优化合并步骤

上面风格化示例中的朴素 BPE 训练实现之所以慢，是因为每做一次合并，它都要遍历所有字节对来找出最高频对。然而，每次合并后发生变化的，只有与被合并对相重叠的那些对的计数。因

此，通过为所有字节对的计数建立索引并增量更新这些计数（而不是显式遍历每一对字节来统计频率），可以提升 BPE 训练的速度。这一缓存手段能带来显著的加速，不过我们要指出，BPE 训练中的合并部分在 Python 中是**无法**并行的。

低资源提示：性能分析（Profiling）

你应当使用 `cProfile` 或 `py-spy` 之类的性能分析工具找出你实现中的瓶颈，并集中优化那些部分。

低资源提示：”缩减规模”（Downscaling）

与其一上来就在完整的 TinyStories 数据集上训练分词器，我们建议你先在一个小的数据子集（”调试数据集”）上训练。例如，你可以改在 TinyStories 验证集上训练分词器——它只有 22K 个文档，而不是 2.12M 个。这体现了一条通用策略：尽可能缩减规模来加速开发，例如使用更小的数据集、更小的模型规模等。选择调试数据集或超参数配置的规模需要仔细权衡：你希望调试集足够大，能够暴露与完整配置相同的瓶颈（这样你所做的优化才能推广到完整配置），但又不能大到运行起来没完没了。

问题（`train_bpe`）：BPE 分词器训练（15 分）

交付要求：编写一个函数，给定输入文本文件的路径，训练一个（字节级）BPE 分词器。你的 BPE 训练函数应至少能处理以下输入参数：

输入

`input_path: str` 存放 BPE 分词器训练数据的文本文件路径。

`vocab_size: int` 一个正整数，定义最终词表的最大尺寸（包括初始字节词表、由合并产生的词表条目以及任何特殊 token）。

`special_tokens: list[str]` 要加入词表的字符串列表。训练期间，把它们视为硬边界，阻止跨越其跨度的合并；但在计算合并统计时不要把它们包含在内。

你的 BPE 训练函数应返回得到的词表和合并：

输出

`vocab: dict[int, bytes]` 分词器词表，即从 `int`（词表中的 token ID）到 `bytes`（token 字节）的映射。

`merges: list[tuple[bytes, bytes]]` 训练产生的 BPE 合并列表。列表的每个元素是一个字节元组 `(<token1>, <token2>)`，表示 `<token1>` 与 `<token2>` 发生了合并。合并应按创建顺序排列。

要用我们提供的测试来检验你的 BPE 训练函数，你需要先实现 `[adapters.run_train_bpe]` 处的测试适配器，然后运行 `uv run pytest tests/test_train_bpe.py`。你的实现应能通过所有测试。可选地（这可能需要投入大量时间），你可以用某种系统级语言来实现训练方法的关键部分，例如 C++（考虑 `cppyy` 或 `nanobind`）或 Rust（使用 PyO3）。如果这样做，请注意哪些操作需要复制

数据、哪些操作可以直接读取 Python 内存，并确保留下构建说明，或者确保仅凭 `pyproject.toml` 就能完成构建。另外请注意，GPT-2 正则表达式在大多数正则引擎中支持不佳，而且在多数支持它的引擎中会很慢。我们验证过 Oniguruma 速度尚可且支持负向前瞻（negative lookahead），不过如果说有什么差别，Python 的 `regex` 包甚至更快。

问题（`train_bpe_tinystories`）：在 TinyStories 上进行 BPE 训练（2 分）

(a) 在 TinyStories 数据集上训练一个字节级 BPE 分词器，最大词表规模取 10,000。确保把 TinyStories 的特殊 token `<|endoftext|>` 加入词表。把得到的词表和合并序列化保存到磁盘，以供进一步检查。训练花费了多少时间和内存？词表中最长的 token 是什么？它合理吗？

资源需求： ≤ 30 分钟（无 GPU）；≤ 30 GB 内存

提示： 在预分词阶段使用 `multiprocessing` 并行化，并利用下面两个事实，你应该能把 BPE 训练的时间控制在 2 分钟以内：

(a) `<|endoftext|>` token 在数据文件中界定文档。

(b) `<|endoftext|>` token 在应用 BPE 合并之前就被作为特殊情况处理。

交付要求： 一到两句话回答。

(b) 对你的代码做性能分析（profile）。分词器训练流程中哪个部分最耗时？

交付要求： 一到两句话回答。

接下来，我们尝试在 OpenWebText 数据集上训练一个字节级 BPE 分词器。与之前一样，我们建议先浏览一下数据集，以更好地理解其内容。

问题（`train_bpe_expts_owt`）：在 OpenWebText 上进行 BPE 训练（2 分）

(a) 在 OpenWebText 数据集上训练一个字节级 BPE 分词器，最大词表规模取 32,000。把得到的词表和合并序列化保存到磁盘，以供进一步检查。词表中最长的 token 是什么？它合理吗？

资源需求： ≤ 12 小时（无 GPU）；≤ 100 GB 内存

交付要求： 一到两句话回答。

(b) 比较对比在 TinyStories 与 OpenWebText 上训练得到的分词器。

交付要求： 一到两句话回答。

2.6 BPE 分词器：编码与解码

在作业的前一部分，我们实现了在输入文本上训练 BPE 分词器的函数，从而得到分词器词表和 BPE 合并列表。现在，我们要实现一个 BPE 分词器：它加载给定的词表和合并列表，并用它们在文本与 token ID 之间进行编码和解码。

2.6.1 编码文本

BPE 编码文本的过程与训练 BPE 词表的过程如出一辙，主要有以下几个步骤。

第 1 步：预分词。我们首先对序列进行预分词，并把每个预分词表示为 UTF-8 字节序列，与 BPE 训练时一样。然后我们在每个预分词内部把这些字节合并为词表条目，各预分词独立处理（不跨越预分词边界进行合并）。

第 2 步：应用合并。然后我们取出 BPE 训练时创建的词表条目合并序列，按其创建顺序依次应用到预分词上。

示例（`bpe_encoding`）：BPE 编码示例

例如，假设输入字符串是'the cat ate'，词表为 `{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}`，学到的合并为 `[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]`。首先，预分词器会把该字符串切分为 `['the', ' cat', ' ate']`。然后我们逐个查看预分词，并应用 BPE 合并。

第一个预分词'the' 初始表示为 `[b't', b'h', b'e']`。查看合并列表，找到第一个适用的合并 (b't', b'h')，用它把预分词变换为 `[b'th', b'e']`。接着回到合并列表，找到下一个适用的合并 (b'th', b'e')，它把预分词变换为 `[b'the']`。最后再回看合并列表，发现没有更多适用的合并了（因为整个预分词已被合并成单个 token），于是 BPE 合并应用完毕。对应的整数序列是 `[9]`。

对其余预分词重复这一过程：预分词' cat' 应用 BPE 合并后表示为 `[b' c', b'a', b't']`，对应整数序列 `[7, 1, 5]`；最后一个预分词' ate' 应用 BPE 合并后为 `[b' at', b'e']`，对应整数序列 `[10, 3]`。因此，编码输入字符串的最终结果是 `[9, 7, 1, 5, 10, 3]`。

特殊 token

在编码文本时，你的分词器应能正确处理用户自定义的特殊 token（在构造分词器时提供）。

内存考量

假设我们想给一个大到无法全部载入内存的文本文件分词。为了高效地给这个大文件（或任何其他数据流）分词，我们需要把它分解为大小可控的块并依次处理每个块，使内存复杂度为常数，而不是随文本大小线性增长。这样做时，必须确保 token 不跨越块边界，否则得到的结果将与把整个序列一次性载入内存分词的朴素方法不同。

2.6.2 解码文本

要把整数 token ID 序列解码回原始文本，只需在词表中查找每个 ID 对应的条目（一个字节序列），把它们拼接起来，再把字节解码为 Unicode 字符串。注意，输入的 ID 不保证能映射为有效的 Unicode 字符串（因为用户可以输入任意整数 ID 序列）。当输入 token ID 无法产生有效的 Unicode 字符串时，你应当把格式错误的字节替换为官方的 Unicode 替换字符 U+FFFD。[3]`bytes.`

[3]关于 Unicode 替换字符的更多信息见 en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character。

`decode` 的 `errors` 参数控制 Unicode 解码错误的处理方式，使用 `errors='replace'` 会自动用替换符替换格式错误的数据。

问题（`tokenizer`）：实现分词器（15 分）

交付要求：实现一个 `Tokenizer` 类：给定词表和合并列表，它能把文本编码为整数 ID，并把整数 ID 解码为文本。你的分词器还应支持用户提供的特殊 token（如果它们尚不在词表中，就把它们附加到词表）。我们推荐如下接口：

```
def __init__(self, vocab, merges, special_tokens=None)
```
用给定的词表、合并列表以及（可选的）特殊 token 列表构造一个分词器。该函数应接受以下参数：

```
vocab: dict[int, bytes]
merges: list[tuple[bytes, bytes]]
special_tokens: list[str] | None = None
```

```
def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)
```
类方法：从序列化的词表和合并列表（格式与你 BPE 训练代码的输出一致）以及（可选的）特殊 token 列表构造并返回一个 `Tokenizer` 。该方法应接受以下附加参数：

```
vocab_filepath: str
merges_filepath: str
special_tokens: list[str] | None = None
```

```
def encode(self, text: str) -> list[int]
```
把输入文本编码为 token ID 序列。

```
def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]
```
给定一个字符串可迭代对象（例如 Python 文件句柄），返回一个惰性产出 token ID 的生成器。对于无法直接载入内存的大文件，要做内存高效分词就必须用到它。

```
def decode(self, ids: list[int]) -> str
```
把 token ID 序列解码为文本。

要让你的 `Tokenizer` 通过我们提供的测试，你需要先实现 `[adapters.get_tokenizer]` 处的测试适配器，然后运行 `uv run pytest tests/test_tokenizer.py` 。你的实现应能通过所有测试。

2.7 实验

问题（`tokenizer_experiments`）：分词器实验（4 分）

(a) 从 TinyStories 和 OpenWebText 中各抽取 10 个文档。使用你先前训练好的 TinyStories 和 OpenWebText 分词器（词表规模分别为 10K 和 32K）把这些抽样文档编码为整数 ID。每个分词器的压缩比（bytes/token）是多少？

交付要求：一到两句话回答。

(b) 如果你用 TinyStories 分词器给 OpenWebText 抽样文档分词，会发生什么？比较压缩比，和/或定性描述出现的现象。

交付要求：一到两句话回答。

(c) 估计你的分词器的吞吐量（例如以 bytes/second 计）。给 Pile 数据集（825GB 文本）分词需要多长时间？

交付要求：一到两句话回答。

(d) 使用你的 TinyStories 和 OpenWebText 分词器，分别把相应的训练集和开发集编码为整数 token ID 序列。我们稍后会用它来训练语言模型。我们建议把 token ID 序列化为数据类型为 `uint16` 的 NumPy 数组。为什么 `uint16` 是合适的选择？

交付要求：一到两句话回答。

3 Transformer 语言模型架构

语言模型以一批整数 token ID 序列为输入（即形状为 `(batch_size, sequence_length)` 的 `torch.Tensor`），返回词表上一个（批量的）归一化概率分布（即形状为 `(batch_size, sequence_length, vocab_size)` 的 PyTorch 张量），其中对每个输入 token 预测的是下一个词的分布。训练语言模型时，我们用这些下一词预测来计算实际的下一词与预测的下一词之间的交叉熵损失。在推理阶段从语言模型生成文本时，我们取最后一个时间步（即序列中的最后一项）预测的下一词分布来生成序列中的下一个 token（例如取概率最高的 token、从分布中采样等），把生成的 token 加入输入序列，然后重复这一过程。

在作业的这一部分，你将从零开始构建这个 Transformer 语言模型。我们先给出模型的高层描述，再逐步细化各个组件。

3.1 Transformer 语言模型

给定一个 token ID 序列，Transformer 语言模型先用输入嵌入（embedding）把 token ID 转换为稠密向量，让嵌入后的 token 依次通过 `num_layers` 个 Transformer 块，最后应用一个可学习的线性投影（" 输出嵌入" 或"LM head"）得到预测的下一 token 的 logit。示意图见图 1。

Token 嵌入

第一步，Transformer 把（批量的）token ID 序列嵌入为一个向量序列，这些向量包含 token 身份的信息（图 1 中的红色方块）。

更具体地说，给定 token ID 序列，Transformer 语言模型用一个 token 嵌入层产生一个向量序列。每个嵌入层接收形状为 `(batch_size, sequence_length)` 的整数张量，产生形状为 `(batch_size, sequence_length, d_model)` 的向量序列。

预归一化（pre-norm）Transformer 块

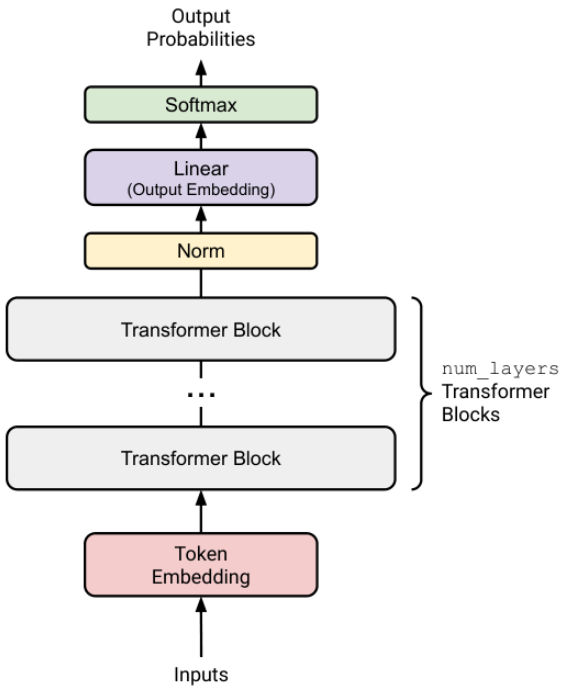


图 1: 我们的 Transformer 语言模型总览。

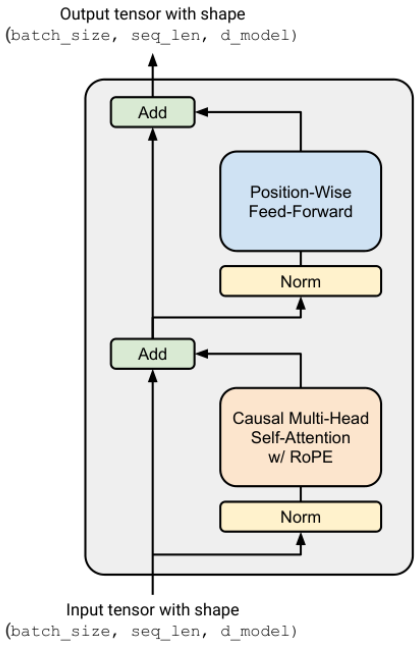


图 2: 预归一化（pre-norm）Transformer 块。

嵌入之后，这些激活值会经过若干结构完全相同的神经网络层。标准的 decoder-only Transformer 语言模型由 `num_layers` 个相同的层（通常称为 Transformer ” 块”）组成。每个 Transformer 块接收形状为 `(batch_size, sequence_length, d_model)` 的输入，并返回形状为 `(batch_size, sequence_length, d_model)` 的输出。每个块通过自注意力在序列维度上聚合信息，并通过前馈层对信息进行非线性变换。

经过 `num_layers` 个 Transformer 块之后，我们取出最终的激活值，把它们转换为词表上的一个分布。

我们将实现” 预归一化（pre-norm）”Transformer 块（详见第 3.4 节），它还要求在最后一个 Transformer 块之后使用层归一化（layer normalization，下文详述），以确保其输出得到恰当的缩放。

这次归一化之后，我们用一个标准的可学习线性变换，把 Transformer 块的输出转换为预测的下一 token 的 logit（例如参见 A. Radford et al. [7] 的式 (2)）。

3.2 附注：批处理、Einsum 与高效计算

在整个 Transformer 中，我们要对许多” 批状” 输入重复执行同样的计算。举几个例子：

- **批内元素**：我们对每个批元素施加相同的 Transformer 前向运算。
- **序列长度**：RMSNorm 和前馈网络这类” 逐位置” 运算对序列的每个位置做完全相同的处理。
- **注意力头**：注意力运算在多个注意力头上批量执行，构成” 多头” 注意力运算。

以一种既充分利用 GPU、又易读易懂的顺手方式执行这类运算，是很有价值的。许多 PyTorch 运算可以在张量开头接受额外的" 批状" 维度，并跨这些维度高效地重复/广播该运算。

例如，假设我们在做一个逐位置的批处理运算。我们有一个形状为 `(batch_size, sequence_length, d_model)` 的" 数据张量" D，想把它与一个形状为 `(d_model, d_model)` 的矩阵 A 做批量向量–矩阵乘法。这时 `D @ A` 会执行批量矩阵乘法——这是 PyTorch 中一个高效的原语，其中 `(batch_size, sequence_length)` 这两个维度被批量化处理。

正因如此，最好假设你的函数可能会被传入额外的批状维度，并把这些维度放在 PyTorch 形状的开头。为了让张量能以这种方式批量处理，可能需要经过多步 `view` 、`reshape` 和 `transpose` 来塑形。这会有点痛苦，而且常常让人难以读懂代码在做什么、张量的形状是什么。

一个更顺手的选择是在 `torch.einsum` 中使用 einsum 记法（einsum notation），或者使用 `einops` 、`einx` 这类框架无关的库。两个关键运算是 `einsum` （可以对任意维度的输入张量做张量缩并）和 `rearrange` （可以重排、拼接和拆分任意维度）。事实证明，机器学习中几乎所有运算都是" 维度搬运" 与张量缩并的某种组合，再加上偶尔出现的（通常是逐点的）非线性函数。这意味着使用 einsum 记法可以让你的许多代码更易读、更灵活。

我们强烈建议本课程的同学学习并使用 einsum 记法。以前没接触过 einsum 记法的同学应该使用 `einops` （见其文档）；已经熟悉 `einops` 的同学应该学习更通用的 `einx` （见其项目页面）。[4]在我们提供的环境中，这两个包都已经安装好了。

下面我们给出一些使用 einsum 记法的例子。它们是 `einops` 文档的补充，你应当先阅读其文档。

示例（`einstein_example1` ）：用 `einops.einsum` 做批量矩阵乘法

```
import torch
from einops import rearrange, einsum

## 基础实现
Y = D @ A.T
# 很难看出输入和输出的形状及其含义。
# D 和 A 可以是什么形状？其中是否存在意外的行为？

## Einsum 是自文档化且稳健的

#                         D                A   ->  Y

Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## 或者，一个批处理版本：D 可以有任意前导维度，但 A 是受限的。
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

[4]值得注意的是，虽然 `einops` 得到的支持非常广泛，但 `einx` 没有经历那么多实战检验。如果你发现 `einx` 有任何限制或 bug，请放心退回使用 `einops` 加一些更普通的 PyTorch。

示例（einstein_example2 ）：用 einops.rearrange 做广播运算

我们有一批图像，想基于某个缩放因子为每张图像生成 10 个变暗版本：

```
images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## 重塑并相乘
dim_value = rearrange(dim_by,    "dim_value          -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## 或者一步到位：
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

示例（einstein_example3 ）：用 einops.rearrange 做像素混合

假设我们有一批图像，表示为形状 (batch, height, width, channel) 的张量，我们想对该图像的所有像素执行一个线性变换，但这个变换对每个通道应当相互独立。该线性变换表示为形状 (height * width, height * width) 的矩阵 B。

```
channels_last = torch.randn(64, 32, 32, 3)  # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## 重排图像张量，以便在所有像素上混合
channels_last_flat = channels_last.view(
   -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

改用 einops 则可以写成：

```
height = width = 32
## rearrange 取代笨拙的 torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
```

```
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)
```

或者，如果你想玩点花的：用 `einx.dot`（`einops.einsum` 的 einx 等价物）一气呵成

```
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

这里的第一种实现可以通过在代码前后加注释来标明输入和输出的形状，但那样很笨拙，而且容易出 bug。使用 einsum 记法，文档即是实现！

Einsum 记法不仅能处理任意的输入批处理维度，还有一个关键好处：自文档化（self-documenting）。使用 einsum 记法编写的代码，能让输入和输出张量的相关形状一目了然。

……在使用 einsum 记法的代码中，输入和输出张量的相关形状会清楚得多。对于其余的张量，你可以考虑使用 Tensor 类型标注，例如使用 jaxtyping 库（它并非 JAX 专属）。

关于使用 einsum 记法对性能的影响，我们会在作业 2 中进一步讨论；但现在你只需要知道：einsum 几乎总是比其他写法更好！

3.2.1 数学记法与内存顺序（Mathematical Notation and Memory Ordering）

许多机器学习论文在记法中使用行向量，这种表示与 NumPy 和 PyTorch 默认采用的行主序（row-major）内存布局十分契合。采用行向量时，线性变换写作

$$y = xW^\top, \tag{1}$$

其中按行主序存储的 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$，行向量 $x \in \mathbb{R}^{1 \times d_{\text{in}}}$。注意，这种写法让我们可以通过增加 x 的最外层维度来对输入做批处理（batching），也就是说，可以把向量输入 x 替换为矩阵输入 $X \in \mathbb{R}^{\text{batch} \times d_{\text{in}}}$。

在线性代数中，更常见的是使用列向量，此时线性变换写作

$$y = Wx, \tag{2}$$

其中 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 按行主序存储，$x \in \mathbb{R}^{d_{\text{in}}}$ 是列向量。在这种设定下若要对输入做批处理，批次维度必须放在 x 的最后，因此 x 需要替换为矩阵 $\tilde{X} \in \mathbb{R}^{d_{\text{in}} \times \text{batch}}$。

在本作业的数学记法中，我们主要使用列向量，因为数学文献通常遵循这一记法。请记住：如果你想使用朴素的矩阵乘法记号，由于 PyTorch 采用行主序内存布局，就必须像式 (1) 的行向量约定那样，在应用矩阵时多做一次转置。如果你的线性代数运算都使用 `einsum` 来写，那么只要轴标签标注正确，这完全不成问题。顺带一提，值得注意的是，其他语言或线性代数软件包（如

Matlab、Julia 和 Fortran）都使用列主序（column-major）内存布局，即批次维度放在最后；而 Python 及其相关软件包采用的是 C 语言标准的行主序布局。

3.3 基础构建模块：Linear 与 Embedding 模块

3.3.1 参数初始化（Parameter Initialization）

要有效地训练神经网络，通常需要仔细地初始化模型参数——糟糕的初始化会导致梯度消失或梯度爆炸等不良行为。Pre-norm Transformer 对初始化出奇地鲁棒，但初始化仍然会显著影响训练速度和收敛性。由于本作业已经很长，我们把细节留到作业 3 再讲，这里直接给出一些在大多数情况下都应该能正常工作的近似初始化方法。目前请使用：

- Linear 权重：$\mathcal{N}\big(\mu = 0,\ \sigma^2 = \frac{2}{d_{\text{in}}+d_{\text{out}}}\big)$，截断（truncated）于 $[-3\sigma, 3\sigma]$。
- Embedding：$\mathcal{N}(\mu = 0,\ \sigma^2 = 1)$，截断于 $[-3, 3]$。
- RMSNorm：1。

你应该使用 `torch.nn.init.trunc_normal_` 来初始化这些截断正态权重。

3.3.2 Linear 模块（Linear Module）

线性层是 Transformer 乃至一般神经网络的基础构建模块。首先，你要实现自己的 `Linear` 类，它继承自 `torch.nn.Module` 并执行线性变换

$$y = Wx. \tag{3}$$

注意：与大多数现代大语言模型一致，我们不包含偏置（bias）项。

问题（linear）：实现 Linear 模块（1 分）

交付要求：实现一个继承自 `torch.nn.Module` 并执行线性变换的 `Linear` 类。你的实现应当遵循 PyTorch 内置 `nn.Linear` 模块的接口，但不带 `bias` 参数（argument）或参数（parameter）。我们推荐如下接口：

`def __init__(self, in_features, out_features, device=None, dtype=None)` 构造线性变换模块。该函数应接受以下参数：

- `in_features: int` ：输入的最终维度。
- `out_features: int` ：输出的最终维度。
- `device: torch.device | None = None` ：参数存储所在的设备。
- `dtype: torch.dtype | None = None` ：参数的数据类型。

`def forward(self, x: torch.Tensor) -> torch.Tensor` 对输入施加线性变换。

请务必做到：

- 继承 `nn.Module` ；
- 调用父类构造函数；
- 把参数构造并存储为 W（而不是 $W^\top$），并放入 `nn.Parameter` ；
- 当然，不要使用 `nn.Linear` 或 `nn.functional.linear` 。

初始化时，请使用上面的设置，并用 `torch.nn.init.trunc_normal_` 初始化权重。

要测试你的 `Linear` 模块，请在 `[adapters.run_linear]` 处实现测试适配器（adapter）。适配器应把给定的权重加载进你的 `Linear` 模块，为此你可以使用 `Module.load_state_dict` 。然后运行 `uv run pytest -k test_linear` 。

3.3.3 Embedding 模块（Embedding Module）

如上所述，Transformer 的第一层是嵌入层（embedding layer），它把整数 token ID 映射到维度为 `d_model` 的向量空间。我们将实现一个继承自 `torch.nn.Module` 的自定义 `Embedding` 类（因此不应使用 `nn.Embedding` ）。`forward` 方法应通过索引形状为 `(vocab_size, d_model)` 的嵌入矩阵，为每个 token ID 选出对应的嵌入向量，其中 token ID 是形状为 `(batch_size, sequence_length)` 的 `torch.LongTensor` 。

问题（embedding）：实现 embedding 模块（1 分）

交付要求：实现继承自 `torch.nn.Module` 并执行嵌入查表（embedding lookup）的 `Embedding` 类。你的实现应当遵循 PyTorch 内置 `nn.Embedding` 模块的接口。我们推荐如下接口：

`def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)` 构造嵌入模块。该函数应接受以下参数：

- `num_embeddings: int` ：词表大小。
- `embedding_dim: int` ：嵌入向量的维度，即 d_{model}。
- `device: torch.device | None = None` ：参数存储所在的设备。
- `dtype: torch.dtype | None = None` ：参数的数据类型。

`def forward(self, token_ids: torch.Tensor) -> torch.Tensor` 为给定的 token ID 查出嵌入向量。

请务必做到：

- 继承 `nn.Module` ；

- 调用父类构造函数；
- 把嵌入矩阵初始化为 `nn.Parameter` ；
- 存储嵌入矩阵时，令 `d_model` 为最终维度；
- 当然，不要使用 `nn.Embedding` 或 `nn.functional.embedding` 。

同样，请使用上面的设置进行初始化，并用 `torch.nn.init.trunc_normal_` 初始化权重。

要测试你的实现，请在 `[adapters.run_embedding]` 处实现测试适配器，然后运行 `uv run pytest -k test_embedding` 。

3.4 Pre-Norm Transformer 块

每个 Transformer 块都有两个子层（sub-layer）：多头自注意力机制和一个逐位置前馈网络（position-wise feed-forward network）（见 A. Vaswani et al., 2017，第 3.1 节）。

在最初的 Transformer 论文中，模型在两个子层各自外围使用残差连接（residual connection），随后做层归一化（layer normalization）。这种架构通常被称为 “post-norm” Transformer，因为层归一化作用在子层的输出上。然而，多项工作发现，把层归一化从每个子层的输出移到每个子层的输入处（并在最后一个 Transformer 块之后额外加一层归一化）可以提升 Transformer 的训练稳定性（T. Q. Nguyen et al., 2019；R. Xiong et al., 2020）——这种 “pre-norm” Transformer 块的可视化表示见图 2。随后，Transformer 块每个子层的输出经由残差连接加回到子层的输入上（A. Vaswani et al. [8]，第 5.4 节）。对 pre-norm 的一个直观理解是：存在一条从输入嵌入直通 Transformer 最终输出、不经过任何归一化的干净的 “残差流（residual stream）”，据称这能改善梯度的流动。这种 pre-norm Transformer 如今已是语言模型的标准配置（例如 GPT-3、LLaMA、PaLM 等），因此我们将实现这一变体。下面我们会逐一介绍 pre-norm Transformer 块的各个组件，并依次实现它们。

3.4.1 均方根层归一化（Root Mean Square Layer Normalization）

最初的 Transformer 实现（A. Vaswani et al. [8]）使用层归一化（J. L. Ba et al., 2016）来归一化激活值。参照 H. Touvron et al. [12]，我们将使用均方根层归一化（root mean square layer normalization，RMSNorm；B. Zhang et al. [13]，式 4）来做层归一化。给定激活值向量 $a \in \mathbb{R}^{d_{\text{model}}}$，RMSNorm 会按下式重新缩放每个激活值 a_i：

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \tag{4}$$

其中 $\text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \varepsilon}$。这里 g_i 是可学习的 “增益（gain）” 参数（总共有 d_{model} 个这样的参数），而 ε 是一个超参数，通常固定为 `e-5` 。

为防止对输入平方时发生上溢，你应该把输入提升（upcast）为 `torch.float32` 。总而言之，你的 `forward` 方法应形如：

```
in_dtype = x.dtype
x = x.to(torch.float32)

# 在此处编写执行 RMSNorm 的代码
...
result = ...

# 以原始 dtype 返回结果
return result.to(in_dtype)
```

问题（rmsnorm）：均方根层归一化（1 分）

交付要求：把 RMSNorm 实现为一个 `torch.nn.Module` 。我们推荐如下接口：

`def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)` 构造 RMSNorm 模块。该函数应接受以下参数：

- `d_model: int` ：模型的隐藏维度。
- `eps: float = 1e-5` ：用于数值稳定性的 epsilon 值。
- `device: torch.device | None = None` ：参数存储所在的设备。
- `dtype: torch.dtype | None = None` ：参数的数据类型。

`def forward(self, x: torch.Tensor) -> torch.Tensor` 处理形状为 `(batch_size, sequence_length, d_model)` 的输入张量，并返回相同形状的张量。

注意：如上所述，请记住在执行归一化之前先把输入提升为 `torch.float32` （之后再降回原始 dtype）。

要测试你的实现，请在 `[adapters.run_rmsnorm]` 处实现测试适配器，然后运行 `uv run pytest -k test_rmsnorm` 。

3.4.2 逐位置前馈网络（Position-Wise Feed-Forward Network）

在最初的 Transformer 论文（A. Vaswani et al. [8] 的第 3.3 节）中，Transformer 前馈网络由两个线性变换组成，中间夹一个 ReLU 激活（$\mathrm{ReLU}(x) = \max(0, x)$）。在那个原始架构中，前馈内层的维度通常是输入维度的 4 倍。

然而，与这一原始设计相比，现代语言模型往往包含两点主要改动：使用另一种激活函数，并采用门控机制（gating mechanism）。具体来说，我们将实现 Llama 3（A. Grattafiori et al., 2024）和 Qwen 2.5（A. Yang et al., 2024）等大语言模型采用的 “SwiGLU” 激活函数，它把 SiLU（常称为 Swish）激活与一种叫做门控线性单元（Gated Linear Unit，GLU）的门控机制结合起来。我们还会仿照 PaLM（A. Chowdhery et al., 2022）以来的大多数现代大语言模型（以及 LLaMA，H. Touvron et al., 2023），省略线性层中有时使用的偏置项。

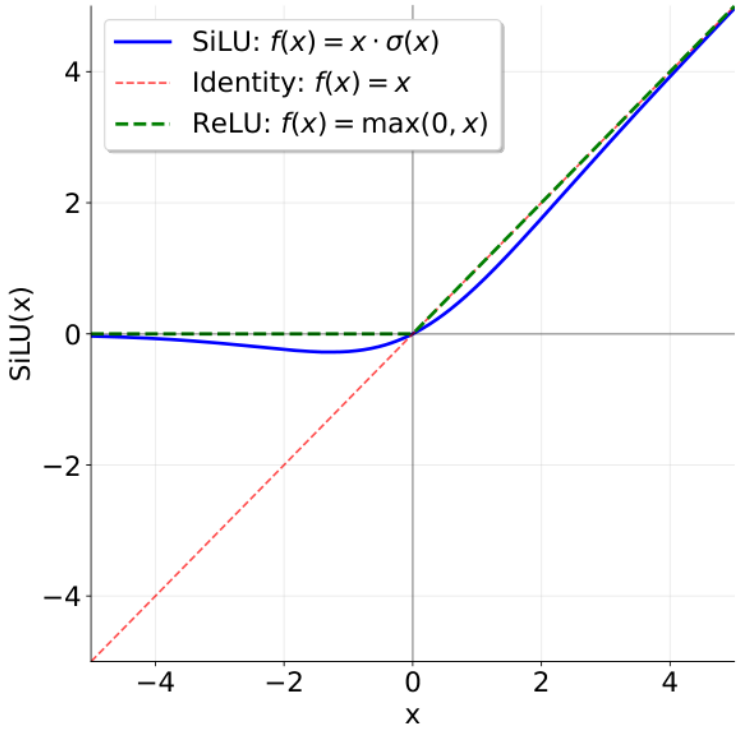


图 3：SiLU（又称 Swish）与 ReLU 激活函数的比较。

SiLU（即 Swish）激活函数（D. Hendrycks et al., 2016；S. Elfwing et al., 2017）定义如下：

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{5}$$

如图 3 所示，SiLU 激活函数与 ReLU 激活函数相似，但在零点处是光滑的。

门控线性单元（GLU）最初由 Y. N. Dauphin et al. [19] 定义为一个经过 sigmoid 函数的线性变换与另一个线性变换的逐元素乘积：

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \tag{6}$$

其中 $\odot$ 表示逐元素乘法。据称门控线性单元能够 “在保留非线性能力的同时，为梯度提供一条线性通路，从而缓解深层架构中的梯度消失问题”。

把 SiLU/Swish 与 GLU 结合起来，就得到 SwiGLU，我们将把它用于我们的前馈网络：

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2\big(\text{SiLU}(W_1 x) \odot W_3 x\big), \tag{7}$$

其中 $x \in \mathbb{R}^{d_{\text{model}}}$，$W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$，$W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$，且规范上取 $d_{\text{ff}} = \frac{8}{3} d_{\text{model}}$。在具体实现中，为了硬件效率，把这个数值取整到邻近的 64 的倍数即可。

N. Shazeer [20] 最先提出把 SiLU/Swish 激活与 GLU 结合，并通过实验表明，在语言建模任务上 SwiGLU 优于 ReLU 和 SiLU（无门控）等基线。在本作业的后面，你也会比较 SwiGLU 和 SiLU。虽然我们已经给出了一些关于这些组件的启发式论证（相关论文也提供了更多支持证据），但保持经验性的视角总是好的：Shazeer 论文中如今很有名的一句话是

> “对于这些架构为何看似有效，我们不给出任何解释；与其他一切原因一样，我们把它们的成功归功于上天的仁慈（divine benevolence）。”

问题（positionwise_feedforward）：实现逐位置前馈网络（2 分）

交付要求：实现由 SiLU 激活函数与 GLU 组成的 SwiGLU 前馈网络。

注意：在这个特定场景中，出于数值稳定性考虑，你可以放心地在实现中使用 `torch.sigmoid`。

在实现中，你应把 d_{ff} 设为约 $\frac{8}{3} \times d_{\text{model}}$，同时确保前馈内层的维度是 64 的倍数，以充分利用你的硬件。要对照我们提供的测试来检验你的实现，你需要在 `[adapters.run_swiglu]` 处实现测试适配器，然后运行 `uv run pytest -k test_swiglu` 来测试你的实现。

3.4.3 相对位置嵌入（Relative Positional Embeddings）

为了向模型注入位置信息，我们将实现旋转位置嵌入（Rotary Position Embeddings，常称 RoPE；J. Su et al., 2021）。对于处于 token 位置 i 的查询 token $q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d$，我们将施加一个成对旋转矩阵 R^i，得到 $q'^{(i)} = R^i q^{(i)} = R^i W_q x^{(i)}$。这里，$R^i$ 会把嵌入元素对 $q^{(i)}_{2k-1:2k}$ 当作二维向量旋转角度

$$\theta_{i,k} = \frac{i}{\Theta^{(2k-2)/d}}, \quad k \in \{1, \ldots, d/2\},$$

其中 Θ 是某个常数。于是，我们可以把 R^i 看作一个 $d \times d$ 的分块对角矩阵，其对角块为 R^i_k（$k \in \{1, \ldots, d/2\}$）：

$$R^i_k = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix} \tag{8}$$

这样我们就得到完整的旋转矩阵

$$R^i = \begin{pmatrix} R^i_1 & 0 & 0 & \cdots & 0 \\ 0 & R^i_2 & 0 & \cdots & 0 \\ 0 & 0 & R^i_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R^i_{d/2} \end{pmatrix}, \tag{9}$$

其中 0 表示 2×2 零矩阵。虽然可以直接构造出完整的 $d \times d$ 矩阵，但一个好的实现应当利用该矩阵的性质来更高效地完成变换。由于我们只关心给定序列内 token 之间的相对旋转，因此对 $\cos(\theta_{i,k})$ 和 $\sin(\theta_{i,k})$ 计算出的值可以在不同层、不同批次之间复用。如果你想进一步优化，可以只用一个 RoPE 模块供所有层引用，并在初始化时用 `self.register_buffer(persistent=False)` 预先创建一个 sin 与 cos 值的二维缓冲区（buffer），而不是 `nn.Parameter`（因为我们不想学习这些固定的余弦和正弦值）。接下来，对我们作用于 $q^{(i)}$ 的完全相同的旋转过程，也要施加于 $k^{(j)}$，即用相应的 R^j 做旋转。注意，这一层没有任何可学习参数。

问题（rope）：实现 RoPE（2 分）

交付要求：实现一个对输入张量施加 RoPE 的 `RotaryPositionalEmbedding` 类。

推荐使用如下接口：

`def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)` 构造 RoPE 模块，并在需要时创建缓冲区。

- `theta: float` ：RoPE 的 Θ 值。
- `d_k: int` ：查询和键向量的维度。
- `max_seq_len: int` ：将输入的最大序列长度。
- `device: torch.device | None = None` ：缓冲区存储所在的设备。

`def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor` 处理形状为 `(..., seq_len, d_k)` 的输入张量并返回相同形状的张量。注意，你应当容忍 x 带有任意数量的批次维度。你应当假定 token 位置是形状为 `(..., seq_len)` 的张量，指定 x 沿序列维度的 token 位置。

你应当使用 token 位置，沿序列维度对你（可能预先计算好的）cos 和 sin 张量做切片。

要测试你的实现，请完成 `[adapters.run_rope]` ，并确保它能通过 `uv run pytest -k test_rope` 。

3.4.4 缩放点积注意力（Scaled Dot-Product Attention）

接下来，我们将按照 A. Vaswani et al. [8]（第 3.2.1 节）的描述实现缩放点积注意力。作为预备步骤，Attention 运算的定义会用到 softmax——一种把未归一化的分数向量变成归一化分布的运算：

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^{n} \exp(v_j)}. \tag{10}$$

注意，当数值较大时 $\exp(v_i)$ 可能变为 inf（此时 $\frac{\text{inf}}{\text{inf}} = \text{NaN}$）。注意到 softmax 运算对 “给所有输入加上同一常数 c” 保持不变，就可以避免这个问题。我们可以利用这一性质来保证数值稳定性——通常做法是把 v 的最大元素从 v 的所有元素中减去，使新的最大元素为 0。接下来你就要实现 softmax，并使用这一技巧来保证数值稳定性。

问题（softmax）：实现 softmax（1 分）

交付要求：编写一个对张量施加 softmax 运算的函数。该函数应接受两个参数：一个张量和一个维度 i，并对输入张量的第 i 个维度施加 softmax。输出张量应与输入张量形状相同，但其第 i 个维度上应是一个归一化的概率分布。请使用 “从第 i 个维度的所有元素中减去该维度最大值” 的技巧，以避免数值稳定性问题。

要测试你的实现，请完成 `[adapters.run_softmax]` ，并确保它能通过 `uv run pytest -k test_softmax_matches_pytorch` 。

现在我们可以从数学上定义 Attention 运算：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{11}$$

其中 $Q \in \mathbb{R}^{n \times d_k}$，$K \in \mathbb{R}^{m \times d_k}$，$V \in \mathbb{R}^{m \times d_v}$。这里 Q、K、V 都是该运算的输入——注意它们不是可学习参数。

掩码（Masking）：有时，对 attention 运算的输出做*掩码*（mask）会很方便。掩码应具有 $M \in \{\text{True}, \text{False}\}^{n \times m}$ 的形状，这个布尔矩阵的第 i 行指示查询 i 应该关注（attend to）哪些键。按照惯例（这有点容易让人混淆），位置 (i, j) 上的 `True` 表示查询 i *确实*关注键 j，而 `False` 表示查询*不*关注该键。换句话说，“信息流动” 发生在值为 `True` 的 (i, j) 位置上。例如，考虑一个元素为 `[[True, True, False]]` 的 1×3 掩码矩阵：这个唯一的查询向量只关注前两个键。

从计算角度看，使用掩码比对子序列分别计算 attention 高效得多；具体做法是取 softmax 之前的值（$QK^\top/\sqrt{d_k}$），并在掩码矩阵为 `False` 的任何位置加上 $-\infty$。

问题（scaled_dot_product_attention）：实现缩放点积注意力（5 分）

交付要求：实现缩放点积注意力函数。你的实现应当处理形状为 `(batch_size, ..., seq_len, d_k)` 的键和查询，以及形状为 `(batch_size, ..., seq_len, d_v)` 的值，其中 `...` 表示任意数量（如果提供的话）的其他类批次（batch-like）维度。实现应返回形状为 `(batch_size, ..., seq_len, d_v)` 的输出。关于类批次维度的讨论见第 3.2 节。

你的实现还应支持一个可选的、由使用者提供的形状为 `(seq_len, seq_len)` 的布尔掩码。掩码值为 `True` 的位置，其注意力概率合计应为 1；掩码值为 `False` 的位置，其注意力概率应为 0。

要对照我们提供的测试来检验你的实现，你需要在 `[adapters.run_scaled_dot_product_attention]` 处实现测试适配器。`uv run pytest -k test_scaled_dot_product_attention` 在三阶输入张量上测试你的实现，而 `uv run pytest -k test_4d_scaled_dot_product_attention` 在四阶输入张量上测试你的实现。

3.4.5 因果多头自注意力（Causal Multi-Head Self-Attention）

我们将按照 A. Vaswani et al. [8] 第 3.2.2 节的描述实现多头自注意力。回顾一下，从数学上讲，多头注意力的运算定义如下：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \tag{12}$$

$$\text{其中} \text{head}_i = \text{Attention}(Q_i, K_i, V_i) \tag{13}$$

其中 Q_i、K_i、V_i 分别是 Q、K、V 的嵌入维度中编号为 $i \in \{1, \ldots, h\}$、大小为 d_k 或 d_v 的切片。这里的 `Attention` 就是第 3.4.4 节定义的缩放点积注意力运算。由此我们可以构造多头自注意力运算：

$$\text{MultiHeadSelfAttention}(x) = W_O \,\text{MultiHead}(W_Q x, W_K x, W_V x) \tag{14}$$

这里，可学习参数是 $W_Q \in \mathbb{R}^{d_k \times d_{\text{model}}}$、$W_K \in \mathbb{R}^{d_k \times d_{\text{model}}}$、$W_V \in \mathbb{R}^{d_v \times d_{\text{model}}}$，以及 $W_O \in \mathbb{R}^{d_{\text{model}} \times d_h}$。由于 Q、K、V 在多头注意力运算中被切片，我们可以把 W_Q、W_K、W_V 看作沿输出维度为每个头分开。当你把这一切跑通时，key、value 和 query 三个投影总共应当只做三次矩阵乘法。[5]

[5]作为更高的挑战目标（stretch goal），可以尝试把 key、query、value 三个投影合并成一个权重矩阵，这样你只需要一次矩阵乘法。

因果掩码（Causal masking）

你的实现应当阻止模型关注序列中未来的 token。换言之，如果模型得到 token 序列 $t_1, \ldots, t_n$，而我们要计算前缀 $t_1, \ldots, t_i$（其中 $i < n$）的下一个词预测，那么模型应当不能访问（关注）位置 $t_{i+1}, \ldots, t_n$ 处的 token 表示，因为在推理阶段生成文本时它本来就拿不到这些 token（而且这些未来的 token 会泄露关于真实下一个词的身份信息，使语言建模预训练目标变得平凡）。对于输入 token 序列 $t_1, \ldots, t_n$，一种朴素的做法是把多头自注意力运行 n 次（对应序列中 n 个不同的前缀），以阻止对未来 token 的访问。但我们将改用因果注意力掩码（causal attention masking），它允许 token i 关注序列中所有满足 $j \leq i$ 的位置 j。你可以使用 `torch.triu` 或广播的索引比较来构造这个掩码，并且应当利用第 3.4.4 节的缩放点积注意力实现已支持注意力掩码这一事实。

应用 RoPE（Applying RoPE）

RoPE 应施加在查询和键向量上，而不是值向量。另外，头（head）维度应当作为批次维度处理，因为在多头注意力中，注意力是对每个头独立施加的。这意味着对每个头的查询和键向量，施加的 RoPE 旋转应当完全相同。

问题（multihead_self_attention）：实现因果多头自注意力（5 分）

交付要求：把因果多头自注意力实现为一个 `torch.nn.Module`。你的实现应当（至少）接受以下参数：

- `d_model: int`：Transformer 块输入的维度。
- `num_heads: int`：多头自注意力中使用的头数。

按照 A. Vaswani et al. [8]，设 $d_k = d_v = \frac{d_{\text{model}}}{h}$。要对照我们提供的测试来检验你的实现，请在 `[adapters.run_multihead_self_attention]` 处实现测试适配器，然后运行 `uv run pytest -k test_multihead_self_attention` 来测试你的实现。

3.5 完整的 Transformer 语言模型

我们先从组装 Transformer 块开始（回顾图 2 会很有帮助）。Transformer 块包含两个“子层”：一个用于多头自注意力，另一个用于 SwiGLU 前馈网络。在每个子层中，我们先执行 RMSNorm，然后执行主运算（MHA/FF），最后加上残差连接。

具体来说，Transformer 块的前半部分（第一个“子层”）应实现下面这组更新，从输入 x 产生输出 y：

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \tag{15}$$

问题（transformer_block）：实现 Transformer 块（3 分）

按照第 3.4 节的描述和图 2 的示意，实现 pre-norm Transformer 块。你的 Transformer 块应当（至少）接受以下参数：

- `d_model: int` : Transformer 块输入的维度。
- `num_heads: int` : 多头自注意力中使用的头数。
- `d_ff: int` : 逐位置前馈内层的维度。

要测试你的实现，请实现适配器 `[adapters.run_transformer_block]` ，然后运行 `uv run pytest -k test_transformer_block` 来测试你的实现。

交付要求：能通过所提供测试的 Transformer 块代码。

现在我们按照图 1 的高层示意图把各个块拼装起来。按照第 3.1.0.1 节对嵌入的描述完成嵌入，把它输入 `num_layers` 个 Transformer 块，然后再送入最后的层归一化和 LM head，得到词表上的未归一化分布（即 logits）。

问题（transformer_lm）：实现 Transformer 语言模型（3 分）

是时候把它们全部组合起来了！请按照第 3.1 节的描述和图 1 的示意实现 Transformer 语言模型。至少，你的实现应当接受 Transformer 块的所有上述构造参数，以及下面这些额外参数：

- `vocab_size: int` : 词表大小，用于确定 token 嵌入矩阵的维度。
- `context_length: int` : 最大上下文长度，用于确定 RoPE 的 sin 和 cos 缓冲区的维度。
- `num_layers: int` : 使用的 Transformer 块数量。

要对照我们提供的测试来检验你的实现，你首先需要在 `[adapters.run_transformer_lm]` 处实现测试适配器，然后运行 `uv run pytest -k test_transformer_lm` 来测试你的实现。

交付要求：能通过上述测试的 Transformer 语言模型模块。

资源核算（Resource accounting）

了解 Transformer 各个部分如何消耗计算和内存是很有用的。我们将逐步完成一些基本的“FLOPs 核算”。Transformer 中的 FLOPs 的绝大部分是矩阵乘法，所以我们的核心方法很简单：

1. 写出 Transformer 前向传播中的所有矩阵乘法。
2. 把每个矩阵乘法换算成所需的 FLOPs。

对于第二步，下面的事实会很有用：

法则：给定 $A \in \mathbb{R}^{m\times n}$ 和 $B \in \mathbb{R}^{n\times p}$，矩阵–矩阵乘积 AB 需要 $2mnp$ 个 FLOPs。

为什么？注意 $(AB)[i,j] = A[i,:]\cdot B[:,j]$，而这个点积需要 n 次加法和 n 次乘法（$2n$ 个 FLOPs）。又因为矩阵–矩阵乘积 AB 有 $m \times p$ 个元素，所以 FLOPs 总数为 $(2n)(mp) = 2mnp$。

在做下一道题之前，建议先把你的 Transformer 块和 Transformer 语言模型的每个组件过一遍，列出所有矩阵乘法及其对应的 FLOPs 开销。

问题（transformer_accounting）：Transformer 语言模型资源核算（5 分）

(a) 考虑一个使用本作业架构的 GPT-2 XL 规模模型，它具有如下配置：

- `vocab_size` ：50,257
- `context_length` ：1,024
- `num_layers` ：48
- `d_model` ：1,600
- `num_heads` ：25
- `d_ff` ：4,288（$\frac{8}{3} \times 1{,}600$ 最接近的 64 的倍数）

假设我们用这个配置构建模型。我们的模型会有多少个可训练参数？假设每个参数用单精度浮点数表示，仅加载这个模型需要多少内存？

交付要求：一到两句话的回答。

(b) 找出完成一次 GPT-2 XL 形状模型前向传播所需的矩阵乘法。这些矩阵乘法总共需要多少 FLOPs？假设输入序列有 `context_length` 个 token。

交付要求：一份矩阵乘法列表（附说明）以及所需 FLOPs 的总数。

(c) 根据你上面的分析，模型的哪些部分需要最多的 FLOPs？

交付要求：一到两句话的回答。

(d) 对 GPT-2 small（12 层，768 `d_model` ，12 头）、GPT-2 medium（24 层，1024 `d_model` ，16 头）和 GPT-2 large（36 层，1280 `d_model` ，20 头）重复你的分析。随着模型规模增大，Transformer 语言模型的哪些部分占总 FLOPs 的比例变大了，哪些变小了？

交付要求：对每个模型，给出模型组件及其对应 FLOPs 的分解（占前向传播所需总 FLOPs 的比例）。此外，用一到两句话描述模型规模的变化如何改变各组件 FLOPs 的占比。

(e) 取 GPT-2 XL 并把上下文长度增加到 16,384。一次前向传播的总 FLOPs 会如何变化？各模型组件 FLOPs 的相对占比会如何变化？

交付要求：一到两句话的回答。

4 训练 Transformer 语言模型

现在我们已有预处理数据（通过分词器）和模型（Transformer）的步骤，剩下的是构建支持训练的全部代码。这包括以下几部分：

- **损失（Loss）：**我们需要定义损失函数（交叉熵）。
- **优化器（Optimizer）：**我们需要定义最小化该损失的优化器（AdamW）。
- **训练循环（Training loop）：**我们需要所有配套的基础设施，用于加载数据、保存检查点（checkpoint）并管理训练。

4.1 交叉熵损失（Cross-entropy loss）

回顾一下，Transformer 语言模型为每条长度为 $m+1$ 的序列 x 和每个 $i=1,\dots,m$ 定义分布 $p_\theta(x_{i+1} \mid x_{1:i})$。给定一个由长度为 $m+1$ 的序列组成的训练集 D，我们定义标准的交叉熵（负对数似然）损失函数：

$$\ell(\theta; D) = \frac{1}{|D|\,m} \sum_{x \in D} \sum_{i=1}^{m} -\log p_\theta(x_{i+1} \mid x_{1:i}). \tag{16}$$

（注意，Transformer 的一次前向传播就能同时得到所有 $i=1,\dots,m$ 的 $p_\theta(x_{i+1} \mid x_{1:i})$。）

具体来说，Transformer 会为每个位置 i 计算 logits $o_i \in \mathbb{R}^{\text{vocab size}}$，由此得到：[6]

$$p(x_{i+1} \mid x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp\big(o_i[x_{i+1}]\big)}{\sum_{a=1}^{\text{vocab size}} \exp(o_i[a])}. \tag{17}$$

交叉熵损失通常是相对于 logits 向量 $o_i \in \mathbb{R}^{\text{vocab size}}$ 和目标 x_{i+1} 定义的。[7]

实现交叉熵损失时需要像 softmax 一样小心处理数值问题。

问题（cross_entropy）：实现交叉熵（1 分）

交付要求：编写一个计算交叉熵损失的函数，它接收预测的 logits（o_i）和目标（x_{i+1}），并计算交叉熵 $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$。你的函数应做到以下几点：

- 为数值稳定性减去最大元素。
- 尽可能地约去 log 和 exp。
- 处理任意额外的批次维度，并返回整个批次的平均值。与第 3.2 节一样，我们假定类批次维度总是排在词表大小维度之前。

请实现 `[adapters.run_cross_entropy]`，然后运行 `uv run pytest -k test_cross_entropy` 来测试你的实现。

困惑度（Perplexity）

交叉熵用于训练已经足够，但在评估模型时，我们还想报告困惑度（perplexity）。对于长度为 m、交叉熵损失依次为 $\ell_1,\dots,\ell_m$ 的序列：

$$\text{perplexity} = \exp\left(\frac{1}{m}\sum_{i=1}^{m} \ell_i\right). \tag{18}$$

4.2 SGD 优化器

有了损失函数之后，我们开始探索优化器。最简单的基于梯度的优化器是随机梯度下降（Stochastic Gradient Descent，SGD）。我们从随机初始化的参数 θ_0 出发，然后在每一步 $t=0,\dots,T-1$ 执行如下更新：

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \tag{19}$$

[6] 注意 $o_i[k]$ 指向量 o_i 中下标为 k 的值。

[7] 这对应于 x_{i+1} 上的 Dirac delta 分布与预测分布 $\text{softmax}(o_i)$ 之间的交叉熵。

其中 B_t 是从数据集 D 中随机采样的一个批次（batch）数据，而学习率（learning rate）α_t 和批次大小（batch size）$|B_t|$ 是超参数。

4.2.1 在 PyTorch 中实现 SGD（Implementing SGD in PyTorch）

要实现我们的优化器，我们将继承 PyTorch 的 `torch.optim.Optimizer` 类。`Optimizer` 子类必须实现两个方法：

`def __init__(self, params, ...)` 应完成优化器的初始化。这里 `params` 是待优化参数的集合（或参数组，以便用户可以对模型的不同部分使用不同的超参数，例如不同的学习率）。务必把 `params` 传给基类的 `__init__` 方法，基类会存储这些参数供 `step` 使用。你可以根据优化器需要接受额外的参数（例如学习率就是一个常见的参数），并把它们以字典的形式传给基类构造函数，字典的键就是你自己为这些参数起的名称（字符串）。

`def step(self)` 应对参数做一次更新。在训练循环中，它会在反向传播之后被调用，因此你可以拿到上一个批次的梯度。该方法应遍历每个参数张量 `p` 并就地（in place）修改它们，也就是设置 `p.data` ——它是与该参数相关联的张量——依据梯度 `p.grad` （如果存在），即损失关于该参数的梯度张量。

PyTorch 的优化器 API 有一些微妙之处，所以用一个例子来讲解会更容易。为了让例子更丰富，我们实现 SGD 的一个小变体：学习率随训练衰减，从初始学习率 α 开始，随时间采取越来越小的步长：

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t) \tag{20}$$

我们来看看这个版本的 SGD 如何实现为一个 PyTorch `Optimizer` ：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"]  # 获取学习率。
            for p in group["params"]:
                if p.grad is None:
                    continue

                state = self.state[p]  # 获取与 p 相关联的状态。
                t = state.get("t", 0)  # 从状态中取出迭代次数，若无则为 0。
```

```
                grad = p.grad.data  # 获取损失关于 p 的梯度。
                p.data -= lr / math.sqrt(t + 1) * grad  # 就地更新权重张量。
                state["t"] = t + 1  # 迭代次数加一。

        return loss
```

在 `__init__` 中，我们把参数以及默认超参数传给基类构造函数（参数可能是分组传入的，每组有各自不同的超参数）。如果参数只是单个 `torch.nn.Parameter` 对象的集合，基类构造函数会创建一个单独的组，并为其指定默认超参数。然后，在 `step` 中，我们遍历每个参数组，再遍历该组中的每个参数，并应用式 (20)。这里，我们把迭代次数作为与每个参数相关联的状态来保存：先读取这个值，在梯度更新中使用它，然后再更新它。该 API 规定，用户可以传入一个可调用对象 `closure` ，在优化器步进之前重新计算损失。对我们将要使用的优化器来说用不到它，但为了符合 API 我们还是加上了。

为了看看它如何工作，我们可以用下面这个*训练循环*（training loop）的最小示例：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad()  # 重置所有可学习参数的梯度。
    loss = (weights**2).mean()  # 计算一个标量损失值。
    print(loss.cpu().item())
    loss.backward()  # 运行反向传播，计算梯度。
    opt.step()  # 运行优化器步进。
```

这就是训练循环的典型结构：在每次迭代中，我们计算损失并执行一步优化器。训练语言模型时，我们的可学习参数来自模型（在 PyTorch 中，`m.parameters()` 会给出这个集合）。损失将在采样出的一个数据批次上计算，但训练循环的基本结构不变。

问题（learning_rate_tuning）：调整学习率（1 分）

正如我们将看到的，对训练影响最大的超参数之一就是学习率。让我们在上面的玩具示例中实际感受一下。把上面的 SGD 示例分别用另外三个学习率 `e1` 、`e2` 和 `e3` 运行，只训练 10 次迭代。每个学习率下损失会发生什么？它衰减得更快、更慢，还是会发散（即随训练进行不断增大）？

交付要求：用一到两句话描述你观察到的行为。

4.3 AdamW

现代语言模型通常不再用 SGD，而是用更复杂的优化器来训练。近来使用的优化器大多源自 Adam 优化器（D. P. Kingma et al., 2015）。我们将使用 AdamW（I. Loshchilov et al., 2019），它在近期工作中被广泛使用。AdamW 对 Adam 提出了一处改进：通过加入*权重衰减*（weight decay，每次迭代把参数向 0 拉近）来改善正则化，而且这种衰减与梯度更新是解耦的。我们将按照 I. Loshchilov et al. [23] 的算法 2 来实现 AdamW。

AdamW 是有状态的（stateful）：它为每个参数维护其第一矩和第二矩的滚动估计。因此，AdamW 用额外的内存换取更好的稳定性和收敛性。除了学习率 α 之外，AdamW 还有一对控制矩估计更新的超参数 (β_1, β_2)，以及一个权重衰减率 λ。一般应用把 (β_1, β_2) 设为 $(0.9, 0.999)$，但 LLaMA（H. Touvron et al., 2023）和 GPT-3（T. B. Brown et al., 2020）这类大语言模型常用 $(0.9, 0.95)$ 来训练。该算法可以写作如下形式，其中 ε 是一个很小的值（例如 10^{-8}），用于在 v 中出现极小值时改善数值稳定性：

算法 1：AdamW 优化器

init(θ) ▷ 初始化可学习参数
$m \leftarrow 0$ ▷ 第一矩向量的初始值；与 θ 形状相同
$v \leftarrow 0$ ▷ 第二矩向量的初始值；与 θ 形状相同
for $t = 1, \ldots, T$ **do**
  采样一个数据批次 B_t
  $g \leftarrow \nabla_\theta \ell(\theta; B_t)$ ▷ 计算损失的梯度
  $\alpha_t \leftarrow \alpha \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}$ ▷ 为第 t 次迭代计算调整后的 α
  $\theta \leftarrow \theta - \alpha\lambda\theta$ ▷ 施加权重衰减
  $m \leftarrow \beta_1 m + (1-\beta_1) g$ ▷ 更新第一矩估计
  $v \leftarrow \beta_2 v + (1-\beta_2) g^2$ ▷ 更新第二矩估计
  $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v}+\varepsilon}$ ▷ 施加经矩调整的权重更新
end for

算法 1：AdamW 优化器伪代码。

注意 t 从 1 开始。接下来你就要实现这个优化器。

问题（adamw）：实现 AdamW（2 分）

交付要求：把 AdamW 优化器实现为 `torch.optim.Optimizer` 的子类。你的类应在 `__init__` 中接收学习率 α，以及 β、ε 和 λ 超参数。为了帮你保存状态，基类 `Optimizer` 为你提供了一个字典 `self.state`，它把 `nn.Parameter` 对象映射到一个字典，用于存储该参数所需的任何信息（对 AdamW 而言就是矩估计）。请实现 `[adapters.get_adamw_cls]`，并确保它能通过 `uv run pytest -k test_adamw`。

问题（adamw_accounting）：AdamW 训练的资源核算（2 分）

我们来计算运行 AdamW 需要多少内存和计算。假设所有张量都使用 `float32`。

(a) 运行 AdamW 需要多少峰值内存？请把答案按参数、激活值、梯度和优化器状态的内存占用分解表示。答案请用 `batch_size` 和模型超参数（`vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads`）表示。假定 $d_{\text{ff}} = \frac{8}{3} \times d_{\text{model}}$。

为简单起见，计算激活值的内存占用时只考虑以下组件：

- Transformer 块
  - RMSNorm（各处）
  - 多头自注意力子层：QKV 投影、$QK^\top$ 矩阵乘法、softmax、对值的加权和、输出投影。
  - 逐位置前馈（SwiGLU）：W_1、W_2、门控分支上的 SiLU、逐元素乘积、W_3

（接上文，计算激活值内存占用时需考虑的组件列表还包括：）

- 最后的 RMSNorm（final RMSNorm）
- 输出嵌入（output embedding）
- 在 logits 上计算交叉熵（cross-entropy on logits）

交付要求：分别给出参数、激活值、梯度和优化器状态各自内存占用的代数表达式，以及总内存占用。

(b) 用 GPT-2 XL 形状的模型实例化你的答案，得到一个只依赖于 `batch_size` 的表达式。在 80GB 内存限制内，你能使用的最大批量大小是多少？

交付要求：形如 $a \cdot \texttt{batch_size} + b$ 的表达式（其中 a、b 为具体数值），以及一个表示最大批量大小的数字。

(c) 运行一步 AdamW 需要多少 FLOPs?

交付要求：一个代数表达式，附简要论证。

(d) 模型 FLOPs 利用率（Model FLOPs utilization，MFU）定义为观测吞吐量（每秒处理的 token 数）与硬件理论峰值 FLOP 吞吐量之比（A. Chowdhery et al. [16]）。一块 NVIDIA H100 GPU 对 “float32”（实际上是 TensorFloat-32，而后者本质上其实是 “bfloat19”）运算的理论峰值为 495 teraFLOP/s。假设你能达到 50% 的 MFU，在单块 H100 上以批量大小 1024 训练 GPT-2 XL 共 400K 步需要多长时间？参照 J. Kaplan et al. [25] 与 J. Hoffmann et al. [26]，假定反向传播的 FLOPs 是前向传播的两倍。

交付要求：训练所需的小时数，附简要论证。

4.4 学习率调度（Learning rate scheduling）

使损失下降最快的学习率取值在训练过程中往往会变化。在训练 Transformer 时，通常的做法是使用学习率调度（schedule）：一开始使用较大的学习率，以便在训练初期进行更快的更新，然后随着模型的训练把它缓慢衰减到一个较小的值。[8]在本作业中，我们将实现训练 LLaMA（H. Touvron et al. [12]）时所使用的余弦退火（cosine annealing）调度。

[8]有时也会使用让学习率重新回升（重启，restarts）的调度，以帮助越过局部极小值。

调度器（scheduler）本质上就是一个函数：它接受当前步数 t 以及其他相关参数（例如初始学习率和最终学习率），并返回第 t 步梯度更新所要使用的学习率。最简单的调度是常数函数：无论 t 取何值，它都返回同一个学习率。

余弦退火学习率调度接受以下参数：(i) 当前迭代步 t；(ii) 最大学习率 $\alpha_{\max}$；(iii) 最小（最终）学习率 $\alpha_{\min}$；(iv) 预热（warm-up）迭代次数 T_w；(v) 余弦退火的最终迭代步 T_c。第 t 次迭代时的学习率定义为：

（预热）若 $t < T_w$，则 $\alpha_t = \frac{t}{T_w}\,\alpha_{\max}$。

（余弦退火）若 $T_w \le t \le T_c$，则 $\alpha_t = \alpha_{\min} + \frac{1}{2}\Big(1 + \cos\Big(\frac{t-T_w}{T_c-T_w}\,\pi\Big)\Big)\,(\alpha_{\max} - \alpha_{\min})$。

（退火后）若 $t > T_c$，则 $\alpha_t = \alpha_{\min}$。

问题（learning_rate_schedule）：实现带预热的余弦学习率调度（1 分）

编写一个函数，它接受 t、$\alpha_{\max}$、$\alpha_{\min}$、T_w 和 T_c，并根据上面定义的调度器返回学习率 α_t。然后实现 `[adapters.get_lr_cosine_schedule]` ，并确保它能通过 `uv run pytest -k test_get_lr_cosine_schedule` 。

4.5 梯度裁剪（Gradient clipping）

在训练过程中，我们有时会遇到产生较大梯度的训练样本，这会使训练变得不稳定。为了缓解这一问题，实践中常用的一种技术是梯度裁剪（gradient clipping）。其思想是在每次反向传播之后、执行优化器步进之前，对梯度的范数施加一个上限。

给定（所有参数的）梯度 g，我们计算它的 ℓ_2 范数 $\|g\|_2$。如果该范数小于最大值 M，则保持 g 不变；否则，我们把 g 按因子 $\frac{M}{\|g\|_2+\varepsilon}$ 缩小（其中加入一个很小的 ε，例如 10^{-6}，以保证数值稳定性）。注意，所得梯度的范数将恰好略低于 M。

问题（gradient_clipping）：实现梯度裁剪（1 分）

编写一个实现梯度裁剪的函数。该函数应接受一个参数列表和一个最大 ℓ_2 范数，并就地（in place）修改每个参数的梯度。使用 $\varepsilon = 10^{-6}$（PyTorch 的默认值）。然后，实现适配器 `[adapters.run_gradient_clipping]` ，并确保它能通过 `uv run pytest -k test_gradient_clipping` 。

5 训练循环（Training loop）

我们现在终于可以把目前为止构建好的主要组件组合在一起了：分词后的数据、模型和优化器。

5.1 数据加载器（Data Loader）

分词后的数据（例如你在 `tokenizer_experiments` 中准备的数据）是单个 token 序列 $x = (x_1, \ldots, x_n)$。即便源数据可能由多个相互独立的文档组成（例如不同的网页或源代码文件），常见的做法也是把它们全部拼接成一个 token 序列，并在文档之间加入分隔符（例如 `<|endoftext|>` token）。

数据加载器（data loader）把这些数据变成一个由批次（batch）组成的流：每个批次由 B 个长度为 m 的序列以及与之配对的相应下一个 token 组成，后者的长度同样为 m。例如，取 $B = 1$、$m = 3$，则 $([x_2, x_3, x_4], [x_3, x_4, x_5])$ 就是一个可能的批次。

以这种方式加载数据能在多个方面简化训练。首先，任意满足 $1 \le i \le n - m$ 的 i 都给出一个有效的训练序列，因此采样训练序列轻而易举。由于所有训练序列的长度都相同，无需对输入序列做填充（padding），这提高了硬件利用率（同时也因为可以增大批量大小 B）。最后，我们也不必为了采样训练数据而加载整个数据集，这使得处理那些原本可能无法全部装入内存的大型数据集变得容易。

问题（data_loading）：实现数据加载（2 分）

交付要求：编写一个函数，它接受一个 numpy 数组 x（由 token ID 组成的整数数组）、一个 `batch_size` 、一个 `context_length` 以及一个 PyTorch 设备字符串（例如 `'cpu'` 或 `'cuda:0'` ），并返回一对张量：采样得到的输入序列，以及与之对应的下一个 token 目标。两个张量的形状都应为 `(batch_size, context_length)` 、包含 token ID，并且都应放置在指定的设备上。要用我们提供的测试来检验你的实现，你需要先实现位于 `[adapters.run_get_batch]` 的测试适配器。然后运行 `uv run pytest -k test_get_batch` 来测试你的实现。

低资源提示：在 CPU 或 Apple Silicon 上加载数据

如果你打算在 CPU 或 Apple Silicon 上训练你的语言模型，就需要把数据移动到正确的设备上（同样地，之后你的模型也应使用相同的设备）。

在 CPU 上，你可以使用 `'cpu'` 设备字符串；在 Apple Silicon（M* 芯片）上，你可以使用 `'mps'` 设备字符串。

关于 MPS 的更多信息，请参考以下资源：

- https://docs.pytorch.org/docs/stable/mps.html
- https://docs.pytorch.org/docs/stable/notes/mps.html
- https://developer.apple.com/documentation/metalperformanceshaders

如果数据集太大而无法全部加载进内存怎么办？我们可以使用名为 `mmap` 的 Unix 系统调用，它把磁盘上的一个文件映射到虚拟内存，并在访问相应内存位置时才惰性地加载文件内容。这样，你就可以“假装”整个数据集都在内存里。Numpy 通过 `np.memmap` （或者，如果你最初是用 `np.save` 保存的数组，也可以给 `np.load` 传入 `mmap_mode='r'` 标志）实现了这一功能：它会返回一个类似 numpy 数组的对象，在你访问条目时按需加载。**在训练期间从你的数据集（即 numpy 数**

组）中采样时，请务必以内存映射模式加载数据集（通过 `np.memmap` ，或视你保存数组的方式给 `np.load` 传入 `mmap_mode='r'` 标志）。另外，请确保指定与所加载数组相匹配的 `dtype` 。显式验证一下内存映射后的数据看起来是否正确（例如，其中不包含超出预期词表大小的值）可能会有帮助。

5.2 检查点（Checkpointing）

除了加载数据之外，我们还需要在训练过程中保存模型。运行作业时，我们常常希望能够恢复一个中途停止的训练运行（例如，作业超时、机器故障等）。即便一切顺利，我们之后可能也想访问中间阶段的模型（例如，事后研究训练动态、从训练的不同阶段采取样本等）。

一个检查点（checkpoint）应当包含恢复训练所需的全部状态。我们当然至少要能恢复模型权重；如果使用有状态的优化器（例如 AdamW），我们还需要保存优化器的状态（对 AdamW 而言即矩估计）；最后，为了恢复学习率调度，我们还需要知道停止时所处的迭代次数。PyTorch 让保存所有这些状态变得很容易：每个 `nn.Module` 都有一个 `state_dict()` 方法，它返回一个包含所有可学习权重的字典；之后我们可以用它的姊妹方法 `load_state_dict()` 恢复这些权重。任何 `torch.optim.Optimizer` 也是如此。最后，`torch.save(obj, dest)` 可以把一个对象（例如一个以若干张量为值的字典，也可以是整数这类普通 Python 对象）转储到一个文件（路径）或类文件对象（file-like object）中，之后再用 `torch.load(src)` 把它加载回内存。

问题（checkpointing）：实现模型检查点（1 分）

实现以下两个函数来加载和保存检查点：

`def save_checkpoint(model, optimizer, iteration, out)` 应把来自模型、优化器和迭代次数的所有状态转储到类文件对象 `out` 中。你可以使用模型和优化器各自的 `state_dict` 方法来获取它们的相关状态，然后用 `torch.save(obj, out)` 把 `obj` 转储到 `out` （PyTorch 在这里既支持路径，也支持类文件对象）。常见的做法是让 `obj` 成为一个字典，但只要之后能够加载你的检查点，使用任何你想要的格式都可以。

该函数需要以下参数：

```
model: torch.nn.Module
optimizer: torch.optim.Optimizer
iteration: int
out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
```

`def load_checkpoint(src, model, optimizer)` 应从 `src` （路径或类文件对象）加载一个检查点，然后从该检查点恢复模型和优化器的状态。你的函数应返回保存到检查点中的迭代次数。你可以使用 `torch.load(src)` 来恢复你在 `save_checkpoint` 实现中保存的内容，并使用模型和优化器上的 `load_state_dict` 方法让它们回到先前的状态。

该函数需要以下参数：

```
src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
model: torch.nn.Module
```

```
optimizer: torch.optim.Optimizer
```

实现 [adapters.run_save_checkpoint] 和 [adapters.run_load_checkpoint] 适配器，并确保它们能通过 uv run pytest -k test_checkpointing 。

5.3 训练循环（Training loop）

现在，终于可以把你实现的全部组件组合进你的主训练脚本了。让你的脚本便于以不同的超参数启动训练运行（例如，通过命令行参数传入超参数）将会带来回报，因为你之后会多次运行它，以研究不同的选择对训练的影响。

问题（training_together）：整合到一起（4 分）

交付要求：编写一个脚本，运行一个训练循环，在用户提供的输入上训练你的模型。特别地，我们建议你的训练脚本（至少）支持以下功能：

- 能够配置和控制各种模型与优化器超参数。
- 使用 np.memmap 以低内存占用的方式加载大规模训练与验证数据集。
- 把检查点序列化到用户提供的路径。
- 定期记录训练与验证性能（例如，输出到控制台和/或像 Weights and Biases 这样的外部服务）。[9]

6 文本生成（Generating text）

既然我们已经能够训练模型了，还差最后一块：从我们的模型生成文本的能力。回顾一下，语言模型接受一个（可能是分批的）长度为 sequence_length 的整数序列，并产生一个大小为 (sequence_length, vocab_size) 的矩阵，其中序列每个位置上的元素是一个预测该位置之后下一个 token 的概率分布。我们现在来编写几个函数，把它变成一个针对新序列的采样方案。

Softmax

按照惯例，语言模型的输出是最后一个线性层的输出（即 “logits”），因此我们需要通过 softmax 运算把它变成归一化的概率。softmax 这个运算我们在前文的式 (10) 中已经见过。

解码（Decoding）

要从我们的模型生成文本（解码，decode），我们会向模型提供一个前缀 token 序列（即 “提示词”，prompt），让它给出词表上的一个概率分布来预测序列中的下一个 token。然后，我们从这个词表分布中采样，以确定下一个输出 token。

[9] wandb.ai

具体而言，解码过程的一步应接受序列 $x_{1..t}$，并通过下式返回一个 token x_{t+1}：

$$P(x_{t+1} = i \mid x_{1..t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)} \tag{21}$$

$$v = \text{TransformerLM}(x_{1..t})_t \in \mathbb{R}^{\texttt{vocab_size}} \tag{22}$$

其中 TransformerLM 是我们的模型：它接受一个长度为 `sequence_length` 的序列作为输入，产生一个大小为 `(sequence_length, vocab_size)` 的矩阵；我们取该矩阵的最后一个元素，因为我们要找的是第 t 个位置上的下一个 token 预测。

反复从这些单步条件分布中采样（并把先前生成的输出 token 追加到下一个解码时间步的输入中），直到生成出序列结束 token `<|endoftext|>`（或达到用户指定的最大生成 token 数），我们就得到了一个基本的解码器。

解码技巧（Decoder tricks）

我们将使用小模型做实验，而小模型有时会生成质量很低的文本。两个简单的解码技巧可以帮助解决这些问题。第一个是*温度缩放*（temperature scaling）：我们在 softmax 中引入一个温度参数 τ，新的 softmax 为

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\texttt{vocab_size}} \exp(v_j/\tau)}. \tag{23}$$

注意，把 τ 设为趋近 0 会使 v 中最大的元素占据主导，softmax 的输出随之变成一个集中在这个最大元素上的 one-hot 向量。

第二个技巧是*核采样*（nucleus sampling）或称 *top-p 采样*：通过截断低概率的 token 来修改采样分布。设 q 是对大小为 `vocab_size` 的（经温度缩放的）softmax 得到的概率分布。带超参数 p 的核采样按下式产生下一个 token：

$$P(x_{t+1} = i \mid q) = \begin{cases} \dfrac{q_i}{\sum_{j \in V(p)} q_j} & \text{若} i \in V(p) \\ 0 & \text{否则} \end{cases} \tag{24}$$

其中 $V(p)$ 是使得 $\sum_{j \in V(p)} q_j \geq p$ 成立的*最小*下标集合。你可以先按大小对概率分布 q 排序，然后不断选取最大的词表元素，直到达到目标水平 p，就能轻松计算出这个集合。

问题（decoding）：解码（3 分）

交付要求：实现一个从你的语言模型解码（生成文本）的函数。我们建议你支持以下特性：

- 为用户提供的提示词生成补全（即，接受某个 $x_{1..t}$，并持续采样补全，直到遇到一个 `<|endoftext|>` token）。
- 允许用户控制生成 token 的最大数量。
- 给定期望的温度值，在采样之前对预测的下一个 token 分布应用 softmax 温度缩放。
- Top-p 采样（A. Holtzman et al. [27]，也称核采样），给定一个用户指定的阈值。

7 实验（Experiments）

现在是时候把所有东西合在一起，在一个预训练数据集上训练（小）语言模型了。

7.1 如何运行实验与交付要求（How to Run Experiments and Deliverables）

理解 Transformer 各架构组件背后设计动机的最好方式，就是亲自动手修改并运行它。没有任何东西能替代亲身实践的经验。

为此，能够**快速地、一致地**做实验，并**记录下你所做的事情**十分重要。为了快速实验，我们将在一个小规模模型（总参数量约 17M）和一个简单数据集（TinyStories）上运行大量实验。为了保持一致，你需要以系统化的方式消融（ablate）组件、改变超参数；为了留存记录，我们会要求你提交一份实验日志，以及与每个实验相关联的学习曲线。

为了能够提交损失曲线，请确保**定期评估验证损失，并同时记录步数和实际运行时间（wall-clock time）**。你可能会发现 Weights and Biases 这类日志基础设施很有用。

问题（experiment_log）：实验日志（3 分）

为你的训练与评估代码搭建一套实验跟踪基础设施，使你能够跟踪各实验相对于梯度步数和实际运行时间的损失曲线。

交付要求：实验所用日志基础设施的代码，以及本节下方各作业题目的实验日志（一份记录你所尝试过的所有事情的文档）。

7.2 TinyStories

我们将从一个非常简单的数据集（TinyStories；R. Eldan et al. [1]）开始：模型在上面训练得很快，我们也能观察到一些有趣的行为。获取该数据集的说明见第 1 节。下面是这个数据集的一个示例。

示例（tinystories_example ）：TinyStories 中的一个示例

（以下为语料原文样例，保留英文。）

Once upon a time there was a little boy named Ben. Ben loved to explore the world around him. He saw many amazing things, like beautiful vases that were on display in a store. One day, Ben was walking through the store when he came across a very special vase. When Ben saw it he was amazed! He said, "Wow, that is a really amazing vase! Can I buy it?" The shopkeeper smiled and said, "Of course you can. You can take it home and show all your friends how amazing it is!" So Ben took the vase home and he was so proud of it! He called his friends over and showed them the amazing vase. All his friends thought the vase was beautiful and couldn't believe how lucky Ben was. And that's how Ben found an amazing vase in the store!

7.2.1 超参数调优（Hyperparameter tuning）

我们会告诉你一些非常基础的超参数作为起点，并请你为其他超参数找到能良好工作的设置。

词表大小（Vocab size）10000。典型的词表大小在几万到几十万之间。你应该改变这个值，观察词表和模型行为如何随之变化。

上下文长度（Context length）256。像 TinyStories 这样的简单数据集可能不需要很长的序列，但对于后面的 OpenWebText 数据，你可能想改变这个值。请尝试改变它，观察其对单次迭代运行时间和最终困惑度（perplexity）的影响。

`d_model` **512。**这比许多小型 Transformer 论文中使用的 768 维略小，但会让一切更快。

`d_ff` **1344。**这大约是 $\frac{8}{3}\,d_{\text{model}}$，同时又是 64 的倍数，这对 GPU 性能有利。

RoPE Θ 参数（theta）10000。

层数与头数 4 层、16 头。这总共会给出约 17M 非嵌入（non-embedding）参数，是一个相当小的 Transformer。

处理的总 token 数（Total tokens processed）327,680,000（你的批量大小 × 总步数 × 上下文长度应约等于该值）。

你应该通过反复试验，为以下其他超参数找到合适的默认值：学习率、学习率预热、其他 AdamW 超参数（$\beta_1, \beta_2, \varepsilon$），以及权重衰减（weight decay）。你可以在 D. P. Kingma et al. [22] 中找到这类超参数的一些典型取值。

7.2.2 整合到一起（Putting it together）

现在，你可以把所有东西组合到一起了：拿到一个训练好的 BPE 分词器，对训练数据集进行分词，然后在你编写的训练循环中运行。**重要提示：**如果你的实现正确且高效，上述超参数应当能在 1 块 B200 GPU 上取得约 20–30 分钟的运行时间。如果你的运行时间要长得多，请检查你的数据加载、检查点保存或验证损失代码是否成为了运行时间的瓶颈，以及你的实现是否正确地进行了批处理（batching）。

7.2.3 调试模型架构的技巧与窍门（Tips and tricks for debugging model architectures）

我们强烈建议你熟练掌握 IDE 内置的调试器（例如 VSCode/Zed），与使用 print 语句调试相比，它会为你节省时间。如果你使用文本编辑器，可以使用 `ipdb` 之类的工具。调试模型架构时，还有一些其他好习惯：

- 开发任何神经网络架构时，一个常见的第一步是**过拟合单个 minibatch**。如果你的实现正确，你应该能很快把训练损失压到接近零。
- 在各个模型组件中设置调试断点，检查中间张量的形状，确保它们符合你的预期。
- 监控激活值、模型权重和梯度的范数，确保它们没有爆炸或消失。

问题（learning_rate）：调整学习率（2 B200 小时）（3 分）

学习率是最重要的待调超参数之一。以你训练好的基础模型为出发点，回答以下问题：

(a) 对学习率做一次超参数扫描（sweep），并报告最终的损失（如果优化器发散，请注明发散）。

交付要求：与多个学习率相对应的学习曲线。请解释你的超参数搜索策略。

交付要求：一个在 TinyStories 上验证损失（每 token）不超过 1.45 的模型。

低资源提示：在 CPU 或 Apple Silicon 上训练几步

如果你是在 `cpu` 或 `mps` 上运行，你应当改为把处理的总 token 数降到 40,000,000，这已足以产生相当流畅的文本。你也可以把目标验证损失从 1.45 放宽到 2.00。

我们在 M4 Max 芯片和 36 GB 内存的机器上，用调好的学习率运行我们的参考实现：批量大小 $\times$ 总步数 $\times$ 上下文长度 $= 32 \times 5000 \times 256 = 40{,}960{,}000$ 个 token，在 `cpu` 上耗时 1 小时 22 分钟，在 `mps` 上耗时 36 分钟。在第 5000 步时，我们取得了 1.80 的验证损失。

另外一些提示：

- 使用 N 个训练步时，我们建议调整余弦学习率衰减调度，使其恰好在第 N 步终止衰减（即达到最小学习率）。
- 使用 `mps` 时，请**不要**使用 TF32 内核，也就是说，不要像对 `cuda` 设备那样设置

```
torch.set_float32_matmul_precision('high')
```

  我们曾尝试在 `mps`（torch 版本 2.9.0）上启用 TF32 内核，发现后端有时会悄悄使用有缺陷的内核，导致训练不稳定。
- 你可以通过 `torch.compile` 对模型进行 JIT 编译来加速训练。具体而言：
  - 在 `cpu` 上，用如下代码编译你的模型：

```
model = torch.compile(model)
```

  - 在 `mps` 上，你可以用如下方式在一定程度上优化反向传播：

```
model = torch.compile(model, backend="aot_eager")
```

  截至 torch 版本 2.9.0，`mps` 尚不支持使用 Inductor 进行编译。

(b) 民间经验认为，最好的学习率位于“稳定性边缘”（edge of stability）。请调查学习率开始发散的临界点与你最佳学习率之间的关系。

交付要求：学习率递增的一系列学习曲线，其中至少包含一次发散的运行，并分析这与收敛速度之间的关系。

现在让我们改变批量大小，看看训练会发生什么。批量大小很重要——它让我们能通过更大的矩阵乘法从 GPU 中获得更高的效率，但批量大小一定是越大越好吗？让我们运行一些实验来一探究竟。

问题（batch_size_experiment）：批量大小变化（1 B200 小时）（1 分）

把你的批量大小从 1 一直变到 GPU 内存上限。中间至少尝试几个批量大小，包括 64 和 128 这样的典型值。

交付要求：不同批量大小下各次运行的学习曲线。如有必要，学习率应当重新优化。

交付要求：用几句话讨论你对批量大小及其对训练影响的发现。

解码器在手，我们就可以生成文本了！我们将从模型生成文本，看看它效果如何。作为参照，你得到的输出至少应当与下面的示例一样好。

示例（`ts_generate_example` ）：TinyStories 语言模型的输出样例

（以下为模型输出原文样例，保留英文。）

Once upon a time, there was a pretty girl named Lily. She loved to eat gum, especially the big black one. One day, Lily's mom asked her to help cook dinner. Lily was so excited! She loved to help her mom. Lily's mom made a big pot of soup for dinner. Lily was so happy and said, "Thank you, Mommy! I love you." She helped her mom pour the soup into a big bowl. After dinner, Lily's mom made some yummy soup. Lily loved it! She said, "Thank you, Mommy! This soup is so yummy!" Her mom smiled and said, "I'm glad you like it, Lily." They finished cooking and continued to cook together. The end.

低资源提示：在 CPU 或 Apple Silicon 上生成文本

如果你使用的是处理了 40M token 的低资源配置，你应该会看到生成结果仍然像英语，但不如上面的流畅。例如，下面是一个在 40M token 上训练的 TinyStories 语言模型的输出样例：

Once upon a time, there was a little girl named Sue. Sue had a tooth that she loved very much. It was his best head. One day, Sue went for a walk and met a ladybug! They became good friends and played on the path together.

"Hey, Polly! Let's go out!" said Tim. Sue looked at the sky and saw that it was difficult to find a way to dance shining. She smiled and agreed to help the talking!"

As Sue watched the sky moved, what it was. She

下面是精确的题目描述以及我们的要求：

问题（generate）：生成文本（1 分）

使用你的解码器和训练好的检查点，报告你的模型生成的文本。你可能需要调整解码器参数（温度、top-p 等）才能得到流畅的输出。

交付要求：至少 256 个 token 的文本转储（或在第一个 `<|endoftext|>` token 处截止），并对该输出的流畅度做简要点评，指出至少两个影响输出好坏的因素。

7.3 消融实验与架构修改（Ablations and architecture modification）

理解 Transformer 的最好方式，就是亲自动手修改它并观察它的行为。我们现在来做几个简单的消融实验和修改。

消融实验 1：层归一化（layer normalization）

人们常说，层归一化对 Transformer 训练的稳定性十分重要。但也许我们想冒险一试。让我们把每个 Transformer 块中的 RMSNorm 都移除，看看会发生什么。

问题（layer_norm_ablation）：移除 RMSNorm 并训练（0.5 B200 小时）（1 分）

从你的 Transformer 中移除所有 RMSNorm 并训练。在之前最优的学习率下会发生什么？使用较低的学习率能获得稳定性吗？

交付要求：移除 RMSNorm 进行训练的学习曲线，以及最佳学习率下的学习曲线。

交付要求：用几句话点评 RMSNorm 的影响。

现在让我们研究另一个初看之下显得随意的层归一化选择。*Pre-norm*（前置归一化）Transformer 块定义为

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)) \tag{25}$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)). \tag{26}$$

这是对原始 Transformer 架构为数不多的 “共识” 修改之一；原始架构使用的是 *post-norm*（后置归一化）方式：

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)) \tag{27}$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)). \tag{28}$$

让我们改回 *post-norm* 方式，看看会发生什么。

问题（pre_norm_ablation）：实现 post-norm 并训练（0.5 B200 小时）（1 分）

把你的 pre-norm Transformer 实现修改成 post-norm 版本。用 post-norm 模型训练，看看会发生什么。

交付要求：post-norm Transformer 的学习曲线，并与 pre-norm 版本进行比较。

我们看到，层归一化对 Transformer 的行为有重大影响，甚至连层归一化的位置都很重要。

消融实验 2：位置嵌入（position embeddings）

接下来，我们将研究位置嵌入对模型性能的影响。具体而言，我们会把基础模型（使用 RoPE）与完全不使用位置嵌入（NoPE）进行比较。事实证明，decoder-only Transformer（即像我们实现的那样带因果掩码的 Transformer）在理论上可以在不显式提供位置嵌入的情况下推断出相对或绝对位置信息（Y.-H. H. Tsai et al. [28]；A. Kazemnejad et al. [29]）。我们现在通过实验来检验 NoPE 与 RoPE 相比表现如何。

问题（no_pos_emb）：实现 NoPE（0.5 B200 小时）（1 分）

修改你带 RoPE 的 Transformer 实现，把位置嵌入信息完全移除，看看会发生什么。

交付要求：比较 RoPE 与 NoPE 性能的学习曲线。

消融实验 3：SwiGLU 与 SiLU（SwiGLU vs. SiLU）

接下来，我们将沿用 N. Shazeer [20] 的思路，通过比较 SwiGLU 前馈网络与使用 SiLU 激活但没有门控线性单元（GLU）的前馈网络的性能，来检验前馈网络中门控的重要性：

$$\mathrm{FFN}_{\mathrm{SiLU}}(x) = W_2\ \mathrm{SiLU}(W_1 x). \tag{29}$$

回顾一下，在我们的 SwiGLU 实现中，我们把内层前馈层的维度设为大约 $d_{\mathrm{ff}} = \frac{8}{3}\, d_{\mathrm{model}}$（同时确保 $d_{\mathrm{ff}} \bmod 64 = 0$，以便利用 GPU 张量核心）。在这个消融基线中，你的 $\mathrm{FFN}_{\mathrm{SiLU}}$ 实现应改为设 $d_{\mathrm{ff}} = 4 \times d_{\mathrm{model}}$，以近似匹配默认 SwiGLU 前馈网络的参数量（后者有三而非两个权重矩阵）。

问题（swiglu_ablation）：SwiGLU 与 SiLU（0.5 B200 小时）（1 分）

交付要求：在参数量近似匹配的前提下，比较 SwiGLU 与 SiLU 前馈网络性能的学习曲线。

交付要求：用几句话讨论你的发现。

低资源提示：GPU 资源有限的线上学员应在 TinyStories 上测试修改

在作业的剩余部分，我们将转向一个更大规模、噪声更多的网页数据集（OpenWebText），试验架构修改，并（可选地）向课程排行榜提交成绩。

在 OpenWebText 上把语言模型训练到流畅需要很长时间，因此我们建议 GPU 资源有限的线上学员继续在 TinyStories 上测试各种修改（以验证损失作为评估性能的指标）。

7.4 在 OpenWebText 上运行（Running on OpenWebText）

我们现在转向一个更标准的预训练数据集，它由网络爬取数据构建。我们还提供了 OpenWebText（A. Gokaslan et al. [2]）的一个小样本，作为一个单独的文本文件：如何获取该文件见第 1 节。

下面是来自 OpenWebText 的一个示例。注意这些文本要真实、复杂、多样得多。你不妨浏览一遍训练数据集，感受一下网络抓取语料的训练数据是什么样子。

示例（owt_example）：OWT 中的一个示例

（以下为语料原文样例，保留英文。）

Baseball Prospectus director of technology Harry Pavlidis took a risk when he hired Jonathan Judge.

Pavlidis knew that, as Alan Schwarz wrote in The Numbers Game, "no corner of American culture is more precisely counted, more passionately quantified, than performances of baseball

players." With a few clicks here and there, you can find out that Noah Syndergaard's fastball revolves more than 2,100 times per minute on its way to the plate, that Nelson Cruz had the game's highest average exit velocity among qualified hitters in 2016 and myriad other tidbits that seem ripped from a video game or science fiction novel. The rising ocean of data has empowered an increasingly important actor in baseball's culture: the analytical hobbyist.

That empowerment comes with added scrutiny – on the measurements, but also on the people and publications behind them. With Baseball Prospectus, Pavlidis knew all about the backlash that accompanies quantitative imperfection. He also knew the site's catching metrics needed to be reworked, and that it would take a learned mind – someone who could tackle complex statistical modeling problems – to complete the job.

"He freaks us out." Harry Pavlidis

Pavlidis had a hunch that Judge "got it" based on the latter's writing and their interaction at a site-sponsored ballpark event. […]

注意：在这个实验中，你可能需要重新调整学习率或批量大小等超参数。

问题（main_experiment）：在 OWT 上做实验（2 B200 小时）（2 分）

在 OpenWebText 上，使用与 TinyStories 相同的模型架构和总训练迭代次数来训练你的语言模型。这个模型表现如何？

交付要求：你的语言模型在 OpenWebText 上的学习曲线。描述损失与 TinyStories 上的差异——我们应该如何解读这些损失？

交付要求：OpenWebText 语言模型生成的文本，格式与 TinyStories 的输出相同。这段文本的流畅度如何？既然模型和计算预算与 TinyStories 相同，为什么输出质量更差？

7.5 你自己的改进 + 排行榜（Your own modification + leaderboard）

恭喜你走到这一步。你马上就要完成了！现在，你将尝试改进 Transformer 架构，看看你的超参数和架构与班上其他同学相比表现如何。

排行榜规则

除以下规则外，没有任何限制：

运行时间（Runtime）：你的提交在 B200 上最多可以运行 45 分钟。如果你使用 SLURM 或 Modal，可能需要在你的提交脚本中自行强制执行这一限制。

数据（Data）：你只能使用我们提供的 OpenWebText 训练数据集。

除此之外，你可以随心所欲。

如果你想寻找一些可以实现的想法，可以看看以下资源：

- 最先进的开源 LLM 系列，例如 Llama 3（A. Grattafiori et al. [14]）或 Qwen 2.5（A. Yang et al. [15]）。

- NanoGPT speedrun 仓库（github.com/KellerJordan/modded-nanogpt），社区成员在上面发布了大量用于“极速通关”（speedrunning）小规模语言模型预训练的有趣修改。例如，一个可以追溯到原始 Transformer 论文的常见修改是把输入嵌入和输出嵌入的权重绑在一起（weight tying）（见 A. Vaswani et al. [8]（第 3.4 节）与 A. Chowdhery et al. [16]（第 2 节））。如果你真的尝试权重绑定，可能需要降低嵌入/LM 头初始化的标准差。

在尝试完整的 45 分钟运行之前，你应当先在 OpenWebText 的一个小子集上或在 TinyStories 上测试这些想法。

需要提醒的是，我们注意到，一些在这个排行榜上表现良好的修改未必能推广到更大规模的预训练。我们将在课程的缩放定律（scaling laws）单元进一步探讨这个问题。

问题（leaderboard）：排行榜（10 B200 小时）（6 分）

你将在上述排行榜规则下训练一个模型，目标是在 0.75 B200 小时内使你的语言模型的验证损失最小化。

交付要求：记录到的最终验证损失、一条清晰标明实际运行时间横轴小于 45 分钟的对应学习曲线，以及对你所做工作的描述。我们期望排行榜提交至少击败 5.0 损失这一朴素基线。在此提交排行榜：github.com/stanford-cs336/assignment1-basics-leaderboard。

参考文献（Bibliography）

[1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?.” 2023.

[2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019.

[3] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proc. of ACL*, 2016.

[4] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.

[5] P. Gage, “A new algorithm for data compression,” *C Users Journal*, vol. 12, no. 2, pp. 23–38, Feb. 1994.

[6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners.” 2019.

[7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.

[8] A. Vaswani *et al.*, “Attention is All you Need,” in *Proc. of NeurIPS*, 2017.

[9] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in *Proc. of IWSLT*, 2019.

[10] R. Xiong *et al.*, “On Layer Normalization in the Transformer Architecture,” in *Proc. of ICML*, 2020.

[11] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016.

[12] H. Touvron *et al.*, “LLaMA: Open and Efficient Foundation Language Models.” 2023.

[13] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in *Proc. of NeurIPS*, 2019.

[14] A. Grattafiori *et al.*, “The Llama 3 Herd of Models.” [Online]. Available: https://arxiv.org/abs/2407.21783

[15] A. Yang *et al.*, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.

[16] A. Chowdhery *et al.*, “PaLM: Scaling Language Modeling with Pathways.” 2022.

[17] D. Hendrycks and K. Gimpel, “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units.” 2016.

[18] S. Elfwing, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” [Online]. Available: https://arxiv.org/abs/1702.03118

[19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” [Online]. Available: https://arxiv.org/abs/1612.08083

[20] N. Shazeer, “GLU Variants Improve Transformer.” 2020.

[21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.

[22] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. of ICLR*, 2015.

[23] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. of ICLR*, 2019.

[24] T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” in *Proc. of NeurIPS*, 2020.

[25] J. Kaplan *et al.*, “Scaling Laws for Neural Language Models.” 2020.

[26] J. Hoffmann *et al.*, “Training Compute-Optimal Large Language Models.” 2022.

[27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in *Proc. of ICLR*, 2020.

[28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, "Transformer Dissection: An Unified Understanding for Transformer's Attention via the Lens of Kernel," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4344–4353. doi: 10.18653/v1/D19-1443.

[29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, "The Impact of Positional Encoding on Length Generalization in Transformers," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. [Online]. Available: https://openreview.net/forum?id=Drrl2gcjzl